Reti di telecomunicazione - Tornatore

Reti Di Telecomunicazioni

Politecnico di Milano

54 pag.

Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

RETI DI TELECOMUNICAZIONE

RITARDI NELLE TRASMISSIONI

Considero una rete semplice, con un collegamento diretto da un punto A ad un punto B: quali sono i contributi di ritardo che ci sono nella trasmissione di un'informazione da A a B? Questo ritardo totale sarà dato dai contributi

$$D = T + \tau$$

- Tempo di trasmissione T
  Il **tempo di trasmissione** è un ritardo dovuto al tipo di "trasmettitore", ovvero è il tempo necessario a mettere l'informazione da trasferire sul mezzo trasmissivo, senza effettivamente inviarla. I bit vengono trasferiti sotto forma di onda e ci vuole del tempo per generarla e metterla sul cavo. Posso definire T_{bit} **tempo di bit** o di trasmissione di un singolo bit, dato da

  $$T_{bit} = \frac{1}{Capacità\ trasmissione}$$

  In generale posso definire il tempo di trasmissione come

  $$T = \frac{L}{C}$$

  dove L è la lunghezza dell'informazione considerata e C è la velocità di trasmissione della sorgente (o trasmettitore generico). Valgono anche le formule inverse per ricavare lunghezza dell'informazione e velocità della sorgente. Più la sorgente è veloce, più il tempo di trasmissione è basso.

- Tempo di propagazione τ
  Una volta che l'informazione è pronta deve essere effettivamente trasferita alla destinazione. Il **tempo di propagazione** è il tempo necessario per trasferire l'informazione attraverso un canale di comunicazione e viene indicato con τ. Viene calcolato come

  $$\tau = \frac{d}{c_x}$$

  Dove d è la distanza in km della trasmissione e c_x è la velocità di propagazione dei segnali elettromagnetici nel mezzo trasmissivo scelto. Se consideriamo la **propagazione libera** (onde radio, vuoto, aria) abbiamo $c_x = c = 3 * 10^8\ km/s$, la velocità della luce.
  Per il materiale generico invece si considera $c_x = \mu * c$, dove μ è l'indice di rifrazione del materiale. Per il rame e la fibra ottica si considera sempre la **propagazione guidata**, con $c_x \cong 2 * 10^8\ km/s$, ovvero $5\mu s$ per percorrere 1km.
  Se diminuisco la distanza diminuisco il tempo di propagazione. Se la trasmissione è più veloce diminuisce il tempo di trasmissione. Se il pacchetto da trasferire è più piccolo avrò un tempo di trasmissione più basso.

Servizi di telecomunicazione

Posso classificare i servizi in base a diversi criteri. In ogni caso per avere il servizio bisogna avere una rete.

- Tipologia di servizio
  - Servizi di base
    Sono i servizi che mettono in comunicazione gli utenti, come il telefono
  - Servizi evoluti
    Sono servizi asimmetrici, che passano attraverso la rete e non necessariamente mettono in comunicazione più utenti tra di loro

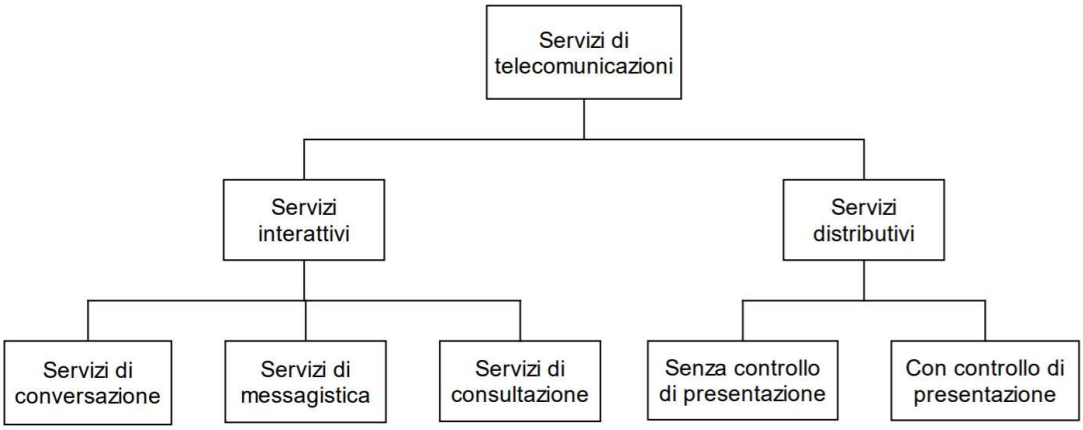
Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

- Tipologia di informazione
  - Voce
  - Immagini (foto e video)
  - Dati (testi, ecc.)

- Tipologia di media
  - Monomediale (usa un solo tipo di informazione)
  - Multimediale (usa più di un tipo di informazione)

- Modalità operative
  - Configurazione
    - Punto-punto (due punti di accesso alla rete)
    - Multipunto (>2 punti di accesso alla rete)
    - Broadcast
  - Direzione
    - Unidirezionale (informazione inviata senza controllo di ricezione
    - Bidirezionale (con conferma, simmetrico come la telefonata o asimmetrico come le pagine web)
  - Inizializzazione
    - Su base chiamata (quando chiamo genero il canale che mi serve)
    - Su base contrattuale (accesso alla rete sempre disponibile o a tempo, è possibile che ci sia congestione nella rete)

Esiste anche una classificazione detta **ITU, International Telecommunication Union**, che regola in modo standard i servizi di telecomunicazione.



- Servizi di conversazione
  Scambio di informazioni in tempo reale (con un ritardo dipendente dal servizio utilizzato). Fa parte di questa categoria la telefonia.
- Servizi di messaggistica
  Consentono lo scambio di informazioni in differita, è necessario l'uso di memorie. Fa parte di questa categoria la posta elettronica.
- Servizi di consultazione
  Consultazione e reperimento di informazioni da centri di servizio. Viene effettuata una richiesta dall'utente ad un centro di servizio e l'utente stesso controlla il trasferimento delle informazioni. Fanno parte di questa categoria il video on demand e la didattica telematica.

Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

- Servizi distributivi senza controllo di presentazione
  Diffusione di informazioni ad un certo numero di utilizzatori, senza controllino l'ordine delle informazioni stesse. Fanno parte di questa categoria la TV e, in generale, la distribuzione audio/video.
- Servizi distributivi con controllo di presentazione
  Diffusione di informazioni ad un certo numero di utilizzatori, facendo in modo che essi controllino l'ordine delle informazioni ricevute (ad esempio televideo).

Requisiti dei servizi
Ogni tipo di servizio ha delle richieste diverse da soddisfare: probabilità di blocco, ritardo, traffico trasportato, probabilità di errore o perdita della comunicazione, ecc.

	Telefonia	Dati interattivi	e-mail	Video-on-demand
Prob. blocco	< 1%	≅ 0	≅ 0	≅ 0
Ritardo medio	< 100 ms	o(1 s)	o(1 min)	o(1 s)
Velocità media	10-64 kbit/s	1-100 kbit/s	o(bit/s)	o(Mbit/s)
Prob. perdita /errore	o(1%)	≅ 0	≅ 0	o(1%)

- **Probabilità di blocco**: un esempio è il sovraccarico di una cella telefonica, nessuno riesce a telefonare
- **Ritardo medio**: in una telefonata, per avere una comunicazione ottimale, si richiede un ritardo medio minore o uguale a 100ms, mentre per altri servizi può essere diverso (specialmente se non sono in tempo reale, come le mail).
- **Velocità media**: è molto importante in servizi che sfruttano una mole di dati istantanea molto grande, come nello streaming video

Caratteristiche delle sorgenti
Esistono sorgenti di due tipologie diverse.
- **Sorgente analogica**: emettono informazioni di vario tipo (suoni, immagini in movimento, ecc.) che devono essere trasformate in segnali elettronici mediante dei trasduttori. I segnali generati da sorgenti analogiche hanno uno spettro continuo, assumono valori arbitrati all'interno di un range di valori
- **Sorgente numerica**: le informazioni emesse sono in forma numerica, formate da sequenze di bit. Questo tipo di informazione ha uno spettro discreto, formato da valori precisi messi in sequenza.

Modellizziamo le sorgenti numeriche con un modello ON-OFF, che presenta alcune grandezze caratteristiche:
- **Frequenza di picco P**
- **Frequenza media A**
- **Tempo di ON T_{ON}**
- **Fattore di burstiness $B = \frac{A}{P}$**

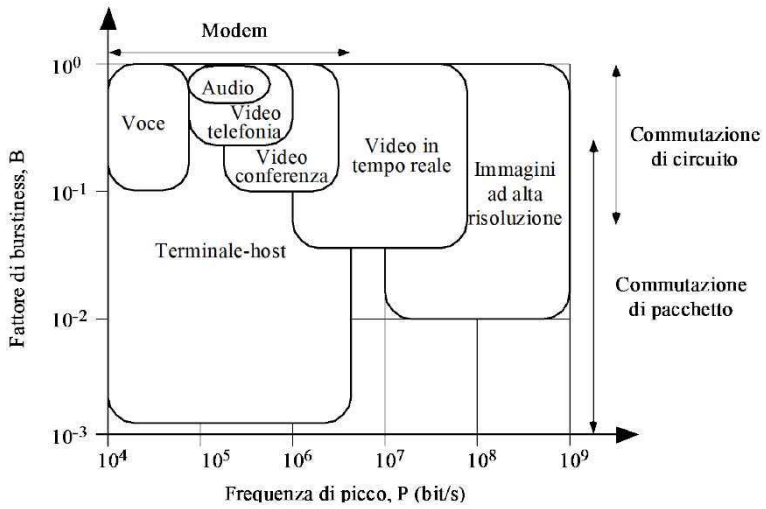
Le sorgenti numeriche possono essere di due tipologie:
- *CBR (Constant Bit Rate)*, sono sorgenti a frequenza (bitrate) costante, senza che venga eliminata nessuna informazione. Classifichiamo nelle sorgenti CBR il trasferimento di file o la codifica di un segnale vocale.
- *VBR (Variable Bit Rate)*, sono sorgenti che non hanno una frequenza costante ma alternano dei momenti di ON e di OFF in modo non definito a priori.

Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

Classifichiamo nelle sorgenti VBR la codifica di un segnale vocale ma con soppressione degli istanti di silenzio, in cui non c'è segnale.

Possiamo definire il fattore di burstiness di una sorgente anche come rapporto dei tempi in cui la sorgente trasmette e quelli in cui non c'è trasmissione, ovvero

$$B = \frac{T_{ON}}{T_{OFF} + T_{ON}}$$



Topologia delle reti

Per rappresentare la topologia delle reti usiamo il formalismo dei grafi. Un grafo è costituito da un insieme di **nodi**, interconnessi tra loro dai **lati**. Per ogni nodo è definito un grado, dato da

$$grado\ nodo = numero\ lati\ incidenti$$

È possibile definire anche un $grado^+$ e un $grado^-$ a seconda del numero di lati uscenti o entranti: questi gradi sono definiti solamente nel caso di un **grafo orientato**, nel quale i lati hanno un verso di percorrenza definito.
Posso definire anche la **lunghezza di un cammino**, dato dalla somma della lunghezza dei lati che formano il cammino necessario per raggiungere un nodo partendo da un altro: esso può essere definito **cammino a costo minimo** se, tra tutti quelli possibili, è quello con il costo minore. Posso definire il **diametro della rete** come il massimo tra tutti i cammini minimi, ovvero il costo del cammino minimo che connette i due nodi più distanti.

Posso distinguere diverse topologie di base a seconda di alcune caratteristiche dei grafi.

- *Rete a maglia*
  Ho N nodi e almeno N+1 link, viene anche detta rete mesh. Con E numero di lati del grafo, vale

  $$N < E < \frac{N(N-1)}{2}$$

  e per il costo del cammino vale $o(N) < costo < o(N^2)$. Se rete completa ho rami dedicati per ogni coppia di nodi e un solo cammino minimo per ogni caso possibile.

- *Rete ad albero*
  Ho N nodi ma N-1 link, senza avere nessun cammino chiuso. I percorsi sono a lunghezza variabile e la rete non è molto affidabile

Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

- *Rete ad anello*
  Ho N nodi, N link ed un solo cammino chiuso. Esiste un solo cammino possibile tra i nodi.
- *Rete a stella*
  N rami dedicati, uno per ogni nodo, con un nodo di commutazione al centro della rete. Ogni percorso ha lunghezza L=2, ma è praticabile solo se i nodi sono fisicamente vicini tra loro.

- *Rete a bus*
  Esiste una struttura centrale che collega tutti i nodi della rete. Vengono attuate procedure di controllo dai nodi, che controllano la comunicazione attraverso il bus. Praticabile solo in area limitata.

- *Rete a maglia+stella*
  Vengono realizzati nodi più complessi, l'instradamento dell'informazioni è complesso

Estensione geografica delle reti
Ogni topologia (albero, anello, maglia, ecc.) viene scelta in base alle dimensioni della rete, perché in alcuni casi è necessario avere misure di sicurezza come la ridondanza.

- *LAN*
  Rete locale (**Local Area Network**), di norma realizzata come rete ad albero per contenere i costi. Per LAN non si intende solo la rete casalinga ma quella di un intero quartiere (alcuni km di cavi).

- *MAN*
  Rete metropolitana (**Metropolitan Area Network**), comprende un'intera citta grande (Milano). Devo avere ridondanza per evitare blackout, poiché la comunicazione tra nodi dipende dai diversi link ed è difficile da ristabilire in tempi brevi. Per una città può essere ad anello (piccola) o a maglia (più grande). Reti fino al centinaio di km di estensione.

- *WAN*
  Rete geografica (**Wide Area Network**), è la rete di un intero paese. Deve essere necessariamente ridondante, comprende migliaia di km di rete.

A prescindere dalla topologia, esistono 3 diverse modalità di utilizzazione dei rami di una rete:
- **Simplex**: mi muovo da un nodo A ad un nodo B ma non posso fare il contrario
- **Duplex**: mi muovo in entrambe le direzioni su un ramo, anche in contemporanea
- **Half-duplex**: mi muovo in entrambe le direzioni ma non contemporaneamente

Internet è una rete distribuita, che interconnette a livello planetario diverse tipologie di rete tra loro.

Modi di trasferimento
Consideriamo aspetti diversi relativi al trasferimento delle informazioni attraverso una rete.

- **Tecnica di commutazione**: come funzionano i nodi quando devono instradare l'informazione
- **Tecnica di multiplazione**: condivisione della banda, varia a seconda della rete di trasporto
- **Architettura protocollare**: come gestire l'accesso multiplo contemporaneo, indirizzamento IP, controllo di flusso/congestione, ecc.

- *Architettura protocollare*
  Ragioniamo secondo raggruppamento e stratificazione: creiamo dei livelli contenenti funzioni simili per logica e/o tecnologia e ordiniamo questi livelli in modo gerarchico, poiché dipenderanno gli uni dagli altri. La "pila" standard viene definita dallo standard ISO-OSI.

Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

Ogni strato (detto **layer**) fornisce un **servizio** al sistema e fa in modo di renderlo disponibile al livello superiore. Ogni layer risolve problemi diversi, rendendoli "invisibili" ai livelli superiori e fornendo solamente le informazioni necessarie al corretto funzionamento del sistema.
Un layer comunica con un layer di un altro sistema ma sullo stesso livello mediante un **protocollo**, mentre comunica con un layer superiore mediante un'**interfaccia**. Due sistemi per rendere possibile la comunicazione tra i propri layer devono avere una **connessione** ad un certo livello.

Sulla pila possono essere effettuati **servizi confermati**, ovvero con la consegna di una conferma da chi effettua la richiesta, sia **servizi non confermati**, che comprendono il solo invio.
Ogni volta che una stringa di bit viene passata al livello superiore vengono aggiunti in testa alcuni bit che specificano le funzionalità del layer inferiore: questa sequenza viene detta **header** e viene utilizzata ogni volta che è necessario aggiungere funzionalità relative ad un livello. L'header viene interpretato dal livello che riceve la stringa.

Con file molto grandi spesso è necessario **segmentare** il file stesso in parti più piccole: gli header vengono aggiunti ad ogni segmento. Quando i vari segmenti giungono a destinazione (oppure ad un livello che riesce a gestire file di dimensioni maggiori) gli header dei singoli segmenti vengono rimossi e il file viene riassemblato.

Didatticamente la pila protocollare viene suddivisa in 5 layer:
- **Applicazione**: fornisce l'interfacciamento con l'utente
- **Trasporto**: ottimizza le prestazioni o colma mancanze della connessione di rete. Trasferisce unità informative dette segmenti ed è il primo strato a compiere multiplazione (TCP).
- **Rete**: ha funzione di trasferimento nodo-nodo, divide le unità informative in pacchetti e li instrada, controllandone il flusso. Non mostra al livello di trasporto le tecniche di commutazione (IP).
- **Collegamento dati**: controlla gli errori nelle unità informative, spezzandole in trame e trasmettendole sequenzialmente gestendo i riscontri ricevuti. Svolge funzione di rilevazione e recupero errori (HDLC).
- **Fisico**: rende disponibile il mezzo fisico per trasferire i singoli bit. Attiva, mantiene e rilascia il circuito fisico necessario per il trasferimento.

Esistono anche altre architetture della pila:
- **Architettura OSI**: applicazione, presentazione (livello di encrypting), sessione (gestiva la comunicazione half duplex). Sotto uguale alla pila precedente.
- **TCP/IP**: applicazione, trasporto, internet (una parte del livello di rete), tutto il resto rientra nel livello di accesso alla rete.

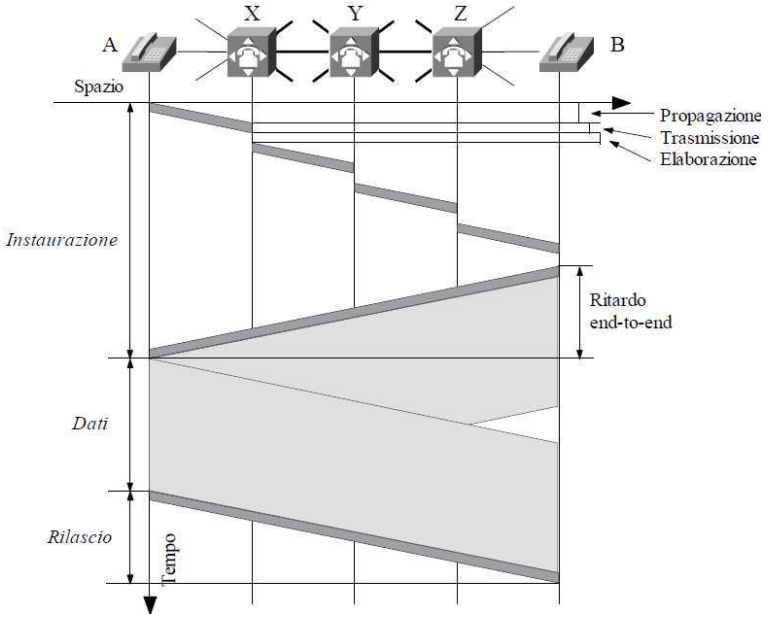
- *Tecniche di commutazione*
La commutazione viene svolta da ogni nodo: consiste nel saper scegliere la strada da percorrere tra due o più link disponibili. È composta da instradamento e attraversamento.
  - **Instradamento (routing)**: svolta a livello software, vengono utilizzati algoritmi opportuni per scegliere il ramo di uscita
  - **Attraversamento (forwarding)**: dipende dall'hardware, attua il passaggio delle unità informative

Esistono due tipologie di commutazione:

- **Commutazione di circuito**: funzionamento della vecchia rete telefonica. Si compone di richiesta, trasferimento e rilascio. Il collegamento viene occupato per tutta la durata della trasmissione, anche se effettivamente non si sta trasmettendo nulla. Per iniziare la trasmissione vengono inviati dei "pacchetti" ai nodi che compongono il circuito in modo da aprirlo e rendere possibile la comunicazione.

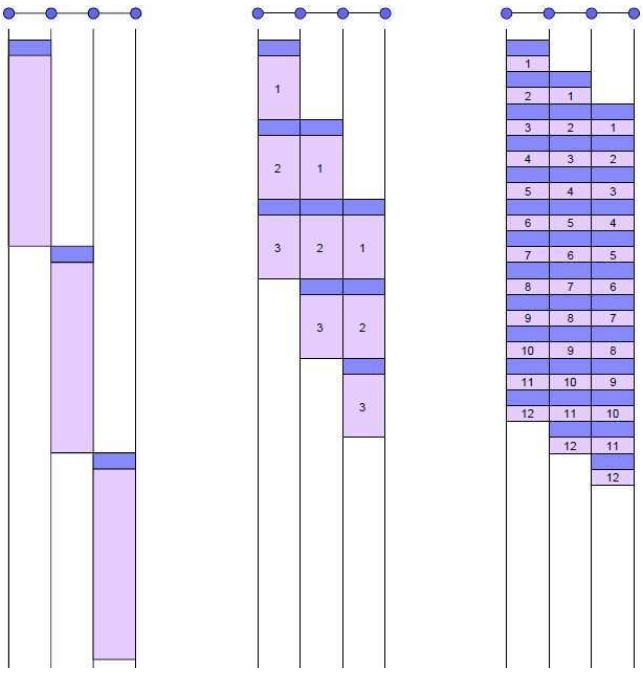
Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

Viene effettuata la comunicazione per intero e, una volta terminata, viene inviato contestualmente un "pacchetto" che informa i nodi che la trasmissione è terminata. Non presenta livelli intermedi, ma solo uno unico.



- **Commutazione di pacchetto**: suddivide l'informazione in pacchetti, occupando ciascuno una quantità di spazio arbitraria. I pacchetti vengono inviati in modo indipendenti e assemblati nuovamente a destinazione. Nella commutazione di pacchetto viene fatto **store and forward**: i pacchetti passano da un buffer per venire successivamente inviati. Ad ogni livello viene aggiunto l'header corrispondente, per poi venire mano a mano rimossi a destinazione per recuperare l'informazione.

  Se suddividessi in troppi pacchetti creerei inutili ritardi: la dimensione degli header è fissa e, in condizioni ideali, il suo tempo di trasmissione è fisso. Se invece suddividessi in pochi pacchetti realizzerei una comunicazione troppo lunga. È necessario trovare un numero di pacchetti ideale per ottimizzare il tempo di trasmissione.



Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

Se nella commutazione di pacchetto non viene bloccato il canale di comunicazione per tutto il tempo necessario, che viene occupato solo per il periodo necessario a trasferire il singolo pacchetto, non è possibile però stabilire a priori il tempo che questo pacchetto impiegherà per arrivare a destinazione.
Inoltre con i pacchetti occuperò momentaneamente un certo spazio in ogni nodo (buffer), mentre nella commutazione di circuito non è necessario.
Se ho un fattore di burstiness alto è più conveniente la commutazione di circuito, mentre al contrario risulta conveniente quella di pacchetto. In generale la commutazione di circuito costa di più rispetto a quella di pacchetto.

Posso definire una commutazione come trasparente: può essere una trasparenza temporale (garantisco sempre lo stesso tempo di trasporto, come la commutazione di circuito), oppure trasparenza semantica (garantisco integrità informativa).

Nella commutazione di pacchetto identifico due tipi di servizi diversi:
- **Servizi connectionless (CL)**: i pacchetti sono trattati in modo indipendente, non viene riservata capacità e possono venirsi a creare dei fuori sequenza.
- **Servizi connetion-oriented (CO)**: viene instaurato un percorso end-to-end, in modo da riservare della capacità e non avere attese muovendosi di nodo in nodo. Quando non trasmetto più da un nodo invio un pacchetto di rilascio in direzione opposta alla comunicazione.

- *Tecniche di multiplazione*
  Multiplazione significa suddividere una connessione ad alta velocità in tante a bassa velocità: vengono combinati più segnali o flussi di dati in un solo segnale multiplato trasmesso in uscita su uno stesso collegamento fisico. Permette di risparmiare sul cablaggio e di ridurre il numero di linee.

  Esiste multiplazione deterministica e multiplazione statistica:
  - Multiplazione deterministica: suddivido la frequenza multiplata f_m in N frequenze di tributario f_t, secondo la formula
    $$f_m = N f_t$$

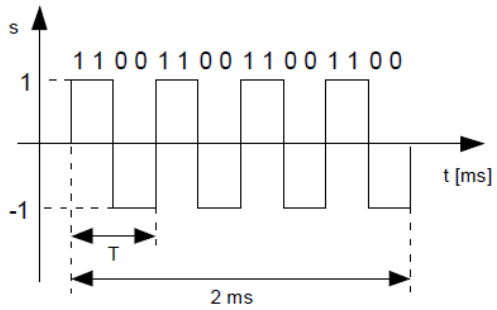
    La linea viene divisa in slot, quindi l'utente la trova sempre completamente libera quando accede. Ogni slot ha una durata pari a
    $$T_s = \frac{L_s}{f_m} = \frac{L_t}{f_t * N} = \frac{T_t}{N} \quad con\ T_t = \frac{L_t}{f_t}$$
  - Multiplazione statistica: tutti gli utenti sono connessi alla linea con una banda totale minore di quella che userebbero trasmettendo tutti in contemporanea. In questo caso deve sempre essere vero che
    $$N * A \leq f_m \leq N * P = N * f_t$$

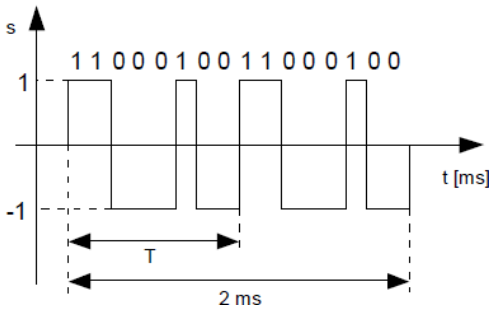
  Nel caso della multiplazione deterministica consideriamo, al fine di dimensionare la connessione, che $C = N * P$. Nel caso della multiplazione statistica invece vale $C = N * A$. Nel primo caso avremo sempre meno utenti ma tutti con la certezza della linea libera, nel secondo molti più utenti ma non è detto che riescano sempre a trasmettere.

Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

Segnali

Tutte le funzioni rappresentate nel tempo possono essere rappresentate, equivalentemente, nel dominio della frequenza. Per semplicità consideriamo segnali periodici.





Un esempio è **l'onda quadra**: in entrambi i casi sopra abbiamo 16 bit rappresentati in 2ms ma con valori differenti nel corso del tempo. Per entrambi il T_{bit} risulta $125\mu s$, ma nel primo il periodo è di $500\mu s$ (invio di 4 bit, vedi grafico), mentre nel secondo il periodo è pari a 1ms (invio di 8 bit). Un qualsiasi segnale periodico può essere rappresentato mediante una somma infinita di seni e coseni: usiamo la serie di Fourier.

Con la **serie di Fourier** si rappresenta ogni segnale periodico come somma di seni e coseni con argomento pari ad un multiplo della frequenza originale (fondamentale) $f_0 = 1/T$.

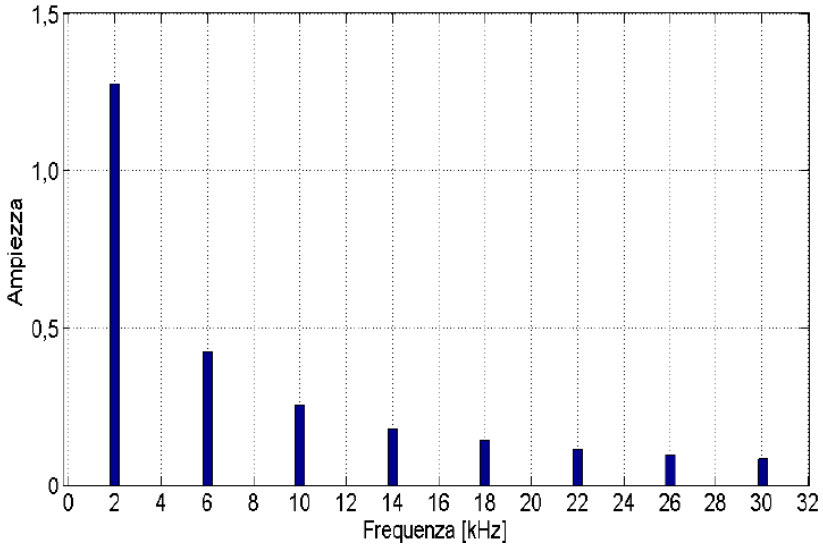
$$s(t) = a_0 + 2\sum_{n=1}^{\infty}\left[a_n \cos(2\pi n f_0 t) + b_n \sin(2\pi n f_0 t)\right]$$

Nel primo dei due casi precedenti vale $f_0 = \frac{1}{500\mu s} = 2kHz$, mentre nel secondo $f_0 = \frac{1}{1ms} = 1kHz$.
La rappresentazione dei segnali precedenti sarà una somma di seni e coseni con multipli di quei valori come argomenti. In ogni caso vale $n \in \mathbb{N}$.
I coefficienti a_n e b_n sono dati dai seguenti integrali

$$a_n = \frac{1}{T}\int_{-T/2}^{T/2} s(t)\cos(2\pi n f_0 t)\,dt \qquad b_n = \frac{1}{T}\int_{-T/2}^{T/2} s(t)\sin(2\pi n f_0 t)\,dt$$

L'onda quadra di periodo T e ampiezza $[+A, -A]$ viene rappresentata, mediante la serie di Fourier, come

$$s(t) = \frac{4A}{\pi}\sum_{n=1,3,5,\ldots,\infty}\frac{\sin(2\pi n f_0 t)}{n}$$

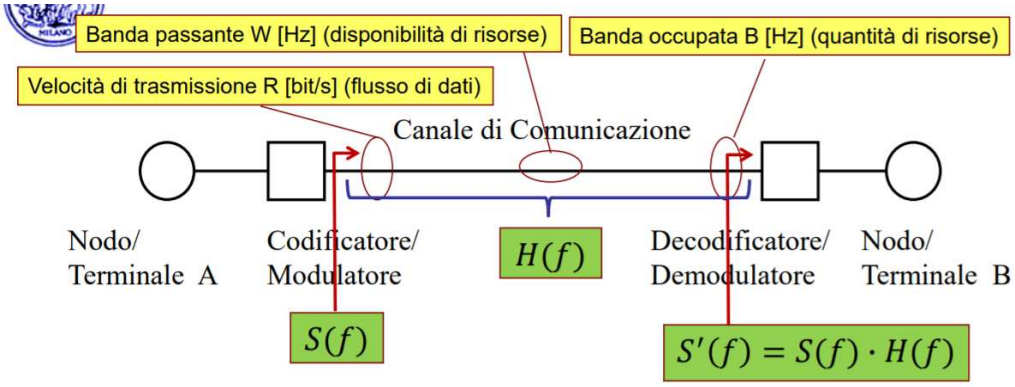


Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

Posso rappresentare graficamente anche lo **spettro del segnale**: sarà la rappresentazione dei pesi delle componenti sinusoidali della serie in base alla loro frequenza.

Per la frequenza di 2kHz avremo un peso abbastanza alto, perché sarà dato da $4/\pi$: il successivo avrà peso $3 * 4/\pi$. Più un segnale è bursty e più lo spettro sarà irregolare, mentre più un segnale è morbido e più lo spettro sarà vicino all'origine e molto regolare (il grafico sopra va già bene).

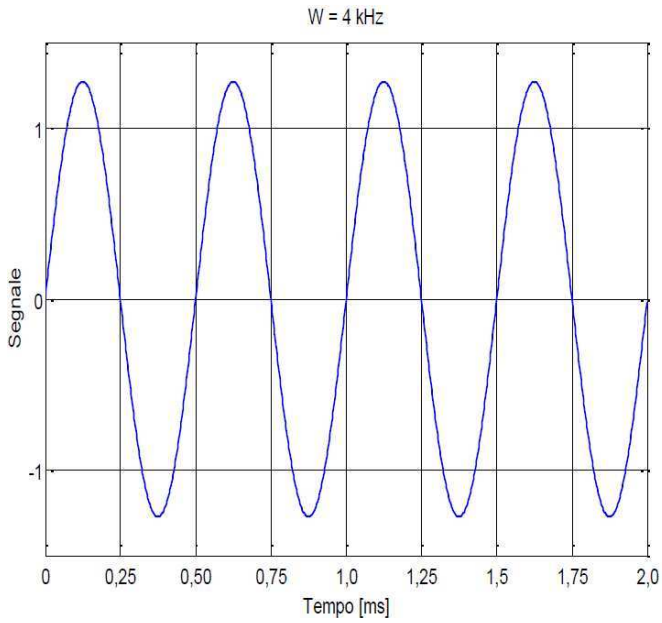
Nella trasmissione di un segnale vediamo che succede quanto segue, per quanto riguarda lo spettro:

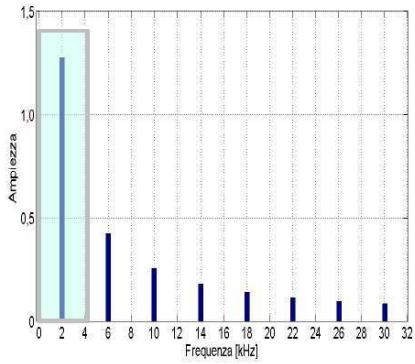


È possibile che sia significativa solamente una parte dello spettro e non tutto: la rappresentazione nelle frequenze della parte significativa corrisponde alla banda occupata dal segnale.
Lo spettro del segnale ricevuto verrà modificato almeno in parte dal canale di comunicazione, poiché $S'(f) = S(f) * H(f)$, dove $S'(f)$ è lo spettro del segnale ricevuto. Il canale deve modificare lo spettro il meno possibile per non distorcere ed è necessario che la banda passante del canale sia maggiore della banda occupata dal segnale.

Tutti i canali di comunicazione sono però limitati, non possono avere banda infinita. In generale, data una banda W, possiamo distinguere due tipologie di canale:

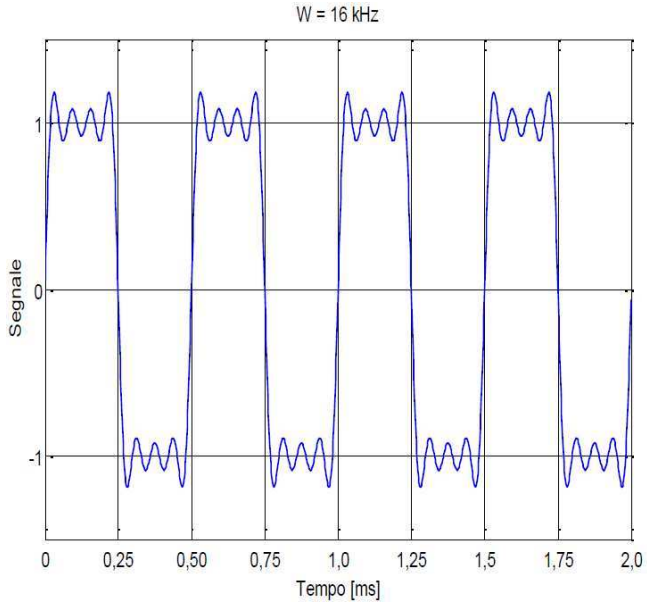
- **Passa-basso**: vengono fatte passare tutte le frequenze comprese in $[0, W]$, ovvero la parte bassa dello spettro del segnale
- **Passa-banda**: vengono fatte passare solo le frequenze comprese in $[f_1, f_2]$, tali che $f_2 - f_1 = W$. Viene selezionata una parte dello spettro da far passare

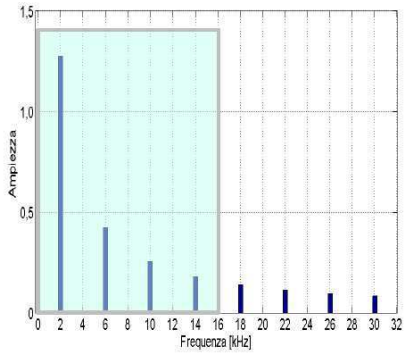




Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

Posso scegliere di limitare il canale come un passa-basso che lasci passare solo le frequenze più basse: con l'esempio precedente otterremmo una sinusoide come in figura. Se scegliessi uno spettro più ampio (non necessariamente utilizzando il canale come passa-banda), otterrei la composizione delle varie frequenze.





- *Conversione tempo-discreto*
  Per poter trasmettere segnali analogici su reti di comunicazione che sfruttano trasmissioni di tipo numerico è necessario operare una conversione: questa operazione viene detta **campionamento**. Trasformiamo un segnale analogico tempo-continuo in un segnale tempo-discreto: generiamo una sequenza di impulsi equidistanti di durata infinitesima ma di ampiezza pari a quella del segnale nell'istante in cui l'impulso viene emesso.

  La distanza tra due campioni viene detta **tempo di campionamento** $\boldsymbol{T_s}$, dal quale deriva la **frequenza di campionamento** $\boldsymbol{f_s = 1/T_s}$, che rappresenta la frequenza con la quale gli impulsi vengono generati. Il segnale originale può essere riscostruito a destinazione senza distorsioni solo in certe condizioni: deve valere il teorema del campionamento.

  Il teorema del campionamento afferma che se il segnale tempo-continuo $s(t)$ è limitato in banda ad una certa frequenza f_{max}, allora per il segnale campionato vale che $s'(kT_s) = s(t)$, con $k \in \mathbb{N}$, solo se per la frequenza di campionamento vale che $\boldsymbol{f_s \geq 2f_{max}}$.

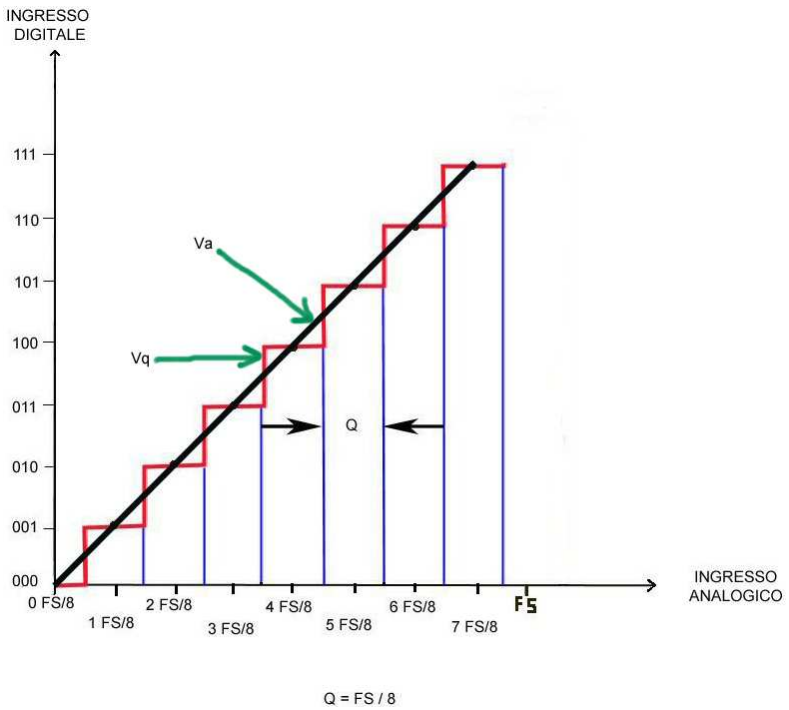
  Può succedere che le frequenze siano talmente tante da non rendere possibile questa operazione: si taglia il segnale ad una certa frequenza, perdendo delle informazioni ma che non sono fondamentali. Usiamo inizialmente un filtro passa-basso (per limitare la banda), campioniamo il segnale, lo inviamo e a destinazione lo ricostruiamo con un altro filtro, che fa in modo di "eliminare" gli artefatti e le copie del campione.

  Oltre che il campionamento devo operare un'altra conversione: dopo aver campionato il segnale ho solamente una sequenza di numeri reali, devo associarli a valori numerici trasmissibili.

  Opero una **quantizzazione**:
- Scelgo un numero di bit sufficiente per rappresentare tutti i valori presenti nel segnale campionato
- Per ogni campione prendo il valore quantizzato più vicino

  Così facendo associo al segnale una sequenza di bit e trasmetto un segnale quantizzato oltre che campionato.

Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

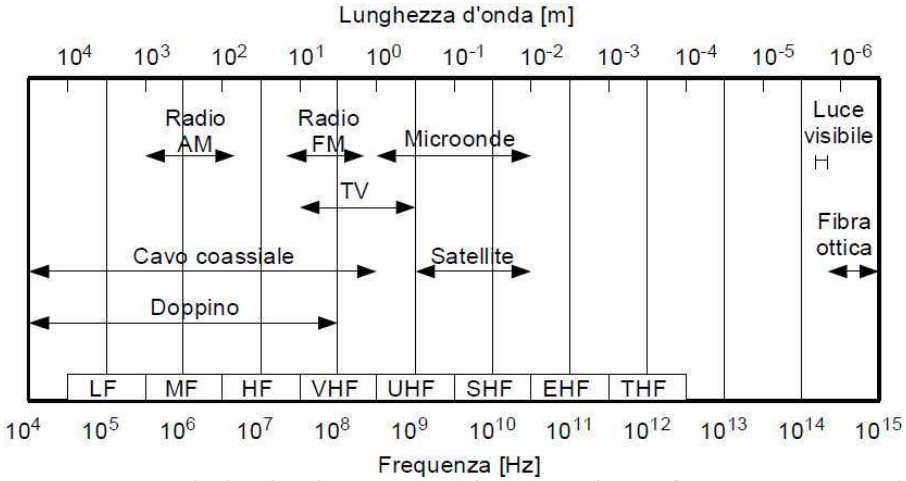


Posso definire anche $\varepsilon = |S_q - S_s|$ **errore di quantizzazione**, ovvero differenza (in valore assoluto) tra il valore quantizzato e il valore reale.

Per il **segnale vocale** ad esempio abbiamo, nel segnale analogico, frequenze fino alle decine di kHz ma la parte "utile" del segnale è quella compresa tra 300Hz e 4kHz: convenzionalmente viene scelta $f_{max} = 4kHz$. Segue che la frequenza di campionamento deve essere almeno $f_s = 2f_{max} = 8kHz$.

Quantizzo il campione con $b = 8\ bit$: significa dividere il campione in $2^8 = 256$ livelli. Posso ricavare anche la frequenza di picco della sorgente vocale facendo $F_c = f_s * b = 8kHz * 8\ bit = 64kbit/s$. Nelle **registrazioni per CD** si utilizza $f_s = 44kHz$ e $b = 16\ bit$, corrispondente a 65536 livelli diversi e ad una frequenza di picco di $F_c = 702\ kbit/s$ per canale, quindi $1{,}4Mbit/s$ per l'audio stereo.

- *<u>Mezzi trasmissivi</u>*



Ciascun mezzo trasmissivo lascia passare solamente alcune frequenze caratteristiche.
Oltre una certa soglia il rame non riesce fisicamente a trasportare informazioni, mentre la fibra trasporta molte più informazioni. La lunghezza d'onda viene definita come $\lambda = v/f$, e solitamente per velocità si prende quella della luce.

Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

- **Cavo coassiale**: ne esistono due tipologie, RG213 e RG58. Sono cavi in rame circondati da una maglia di alluminio e rivestiti in teflon per schermare. RG213 (thick) ha attenuazione più bassa di RG58 (thin).

- **Doppino**: due cavi in rame intrecciati, ne esistono 3 tipi
  - *UTP (Unshielded Twisted Pair)*, è quello della cornetta del telefono e non viene usato in ambito business
  - *STP (Shielded Twisted Pair)*, cavo schermato internamente con schermatura su ogni cavo
  - *FTP (Foiled Twisted Pair)*, solo schermatura esterna

  Il cavo RJ45 classico ha 4 doppini all'interno e ha velocità di trasferimento variabile a seconda delle categorie.

- **Fibra ottica**: formata da due cilindri concentrici in vetro, quello interno ha indice di rifrazione diverso da quello esterno. Quello esterno è detto cladding, quello esterno core.

  Esistono due tipologie:
  - *Monomodo (single mode)*: core molto piccolo, la luce si propaga in un solo modo e corrisponde ad un'unica soluzione delle equazioni di Maxwell. Bassissima attenuazione, viene usata per grandi distanze ma è costosa
  - *Multimodo (multi mode)*: core più grande, trasporta la luce in più modi e porta a più soluzioni delle equazioni di Maxwell. Meno costosa, viene usata su distanze brevi a causa dell'attenuazione.

<u>Reti in area geografica</u>
Per rete in area geografica (WAN) consideriamo reti con un'estensione nell'ordine delle centinaia (o anche migliaia) di km e considereremo solamente la tipologia di rete che sfrutta la commutazione di pacchetto.
Solitamente questo tipo di rete ha una **topologia a maglia incompleta**, data la complessità della rete stessa, e non è presente una gerarchia: esistono nodi di transito (che mettono in comunicazione altri nodi) e nodi di accesso (che interfacciano gli utenti con la rete).

A seconda del livello considerato l'unità informativa assume nomi diversi: consideriamo il livello 2, cioè il meccanismo di scambio informazioni tra nodi adiacenti, e chiamiamo l'unità informativa **frame** (trama), anche se non formalmente viene chiamata sempre pacchetto.

L'obiettivo del livello 2 è di migliorare le prestazioni di un collegamento fisico con un certo tasso di errore, offrendo al livello di rete (livello 3) un servizio di rete privo di errori.

- *<u>Funzioni del livello 2</u>*
- **Gestione del collegamento dati**: rende possibile lo scambio di trame
- **Delimitazione delle trame**: ad ogni trama vengono aggiunti opportuni campi che fanno identificare primo e ultimo bit
- **Indirizzamento**: ogni trama contiene un campo che specifica la destinazione
- **Controllo degli errori**: esistono due strategie

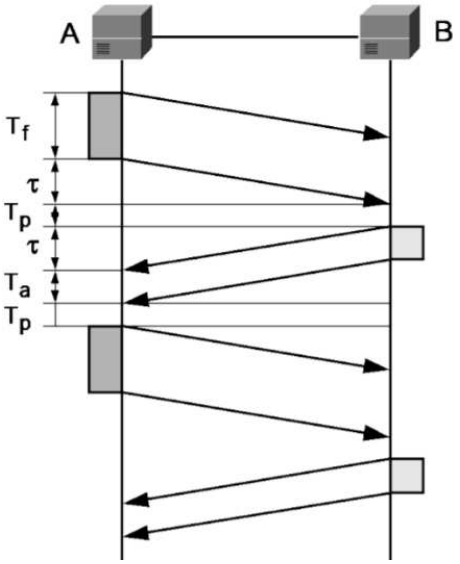
  - FEC (Forward Error Correction): vengono aggiunti bit ridondanti per la correzione degli errori, aumenta la mole di dati trasmessi e tutti devono adottarlo per essere efficace
  - ARQ (Automatic Repeat reQuest): aggiunge un codice di rivelazione errore e nel caso chiede la ritrasmissione della trama, senza correggere nulla. È la più diffusa.

Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

- **Controllo di flusso**: controlla che non ci siano fenomeni di saturazione che fanno perdere trame

- *Automatic Repeat reQuest (ARQ)*
  Esistono differenti protocolli ARQ, ciascuno con le sue peculiarità:
  - ARQ Stop&Wait
  - Continuous ARQ
    - Go-Back-n (GBN)
    - Selective Repeat (SR)

- *ARQ Stop&Wait privo di errori*
  A invia una trama, che viene propagata fino a B, che la elabora. A attende un riscontro da B (acknowledge) prima di trasmettere la trama successiva.



$$T_{tot} = N_f\left(T_f + T_a + 2T_p + 2\tau\right) = \\ = N_f\left(\frac{L_f}{C} + \frac{L_a}{C} + 2T_p + 2\tau\right)$$

N_f: numero di trame
T_f: tempo di trasmissione di una trama (s)
T_a: tempo di trasmissione di un riscontro (s)
T_p: tempo di elaborazione (s)
τ: tempo di propagazione (s)
C: capacità del canale (bit/s)
L_f: lunghezza della trama (bit)
L_a: lunghezza del riscontro (bit)
d: distanza trasmettitore-ricevitore (m)
v: velocità di propagazione (m/s)

Stop&Wait ha il problema dell'attesa: **non occupo mai tutto il tempo che ho a disposizione**. Posso calcolare il tempo di propagazione normalizzato come il rapporto tra il tempo di propagazione e il tempo di trasmissione della trama (di lunghezza costante L_f)

$$a = \frac{\tau}{T_f} = \frac{d/v}{L_f/C} = \frac{dC}{L_f v}$$

L'efficienza di S&W, in assenza di errori, si può calcolare come

$$\eta = \frac{T_f}{T_f + T_a + 2T_p + 2\tau} \cong \frac{T_f}{T_f + 2\tau} = \frac{1}{1 + 2a}$$

Se le trame non fossero a lunghezza costante l'efficienza è pari a

$$\eta = \frac{tempo\ impiegato\ in\ trasmissione}{tempo\ completamento\ trasmissione}$$

Possiamo definire anche l'efficienza di utilizzazione del collegamento (**throughput**) come

$$THR = \eta * C$$

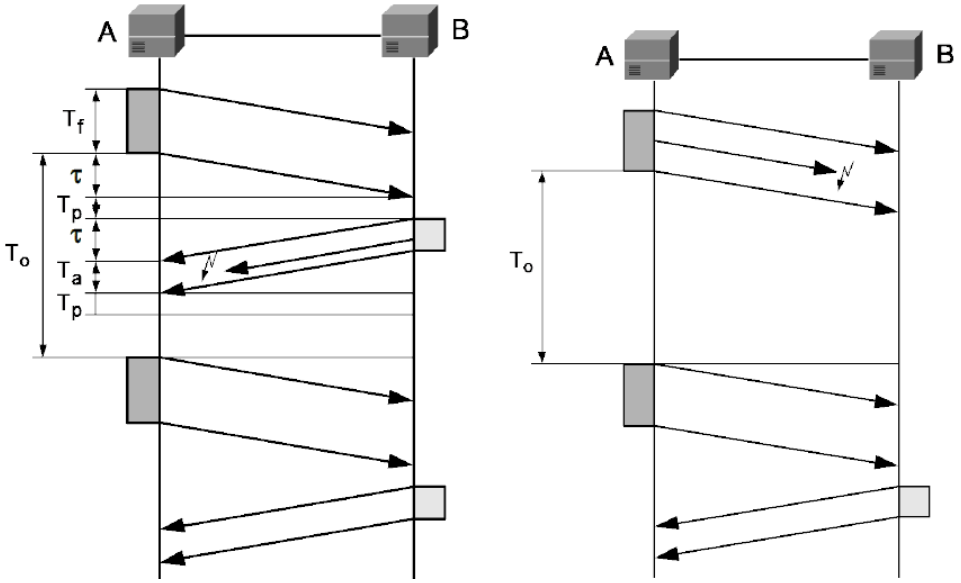
Dove C è la capacità del collegamento: rappresenta quanto uso realmente della capacità totale disponibile.

Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

[PER ESEMPI DI ESERCIZI VEDI QUADERNO]

- *ARQ Stop&Wait in presenza di errori*

  Nei protocolli ARQ gli errori vengono solamente rilevati ma non corretti, richiedendo un nuovo invio della trama corretta. È necessario definire un meccanismo di **timeout** che stabilisce quando inviare nuovamente la trama non riscontrata correttamente (che viene scartata dal ricevente). L'errore può essere presente sia nella trama che nel suo riscontro, quindi è possibile che sia necessario ritrasmettere quest'ultimo.
  Definisco il tempo di timeout come $\boldsymbol{T_O \geq T_a + 2T_p + 2\tau}$: se scegliessi un tempo di timeout minore sarei sempre in timeout e continuerei ad inviare sempre la stessa trama.

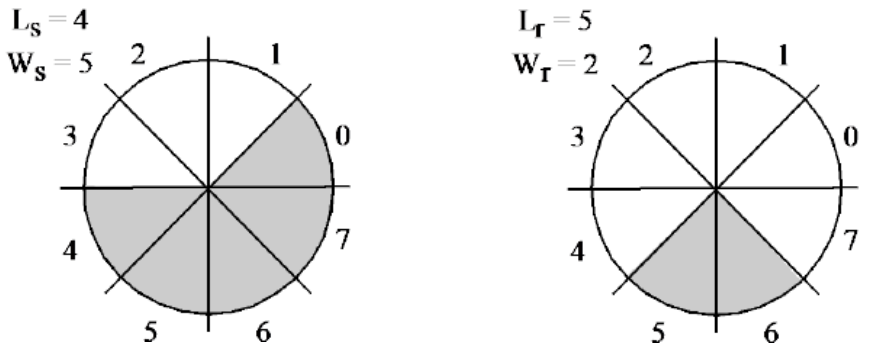


  Se il tempo di elaborazione e il tempo di trasmissione del riscontro risultano trascurabili ($T_a = T_p \cong 0$), posso definire il tempo di timeout minimo come $T_O \geq 2\tau$.
  È necessario, per evitare doppie trasmissioni, numerare le trame con un sistema di numerazione ad 1 bit: alternativamente si assegna alle trame 1 o 0, così facendo il ricevente deve ricevere trame con numerazione alternata, scartando una trama con la stessa numerazione della precedente.

  Per lo stesso motivo è necessario numerare alternativamente anche i riscontri: se per qualche motivo un acknowledge viene inviato dopo il timeout è possibile che arrivi a destinazione nel momento sbagliato (ordine diverso) oppure che non arrivi del tutto. Numerandoli è possibile sapere quale riscontro è problematico. Con acknowledge molto grandi l'efficienza del protocollo cala molto velocemente.

- *Continuous ARQ*
  Per aumentare l'efficienza è necessario eliminare l'attesa del riscontro: è necessario **numerare tutte le trame** con una numerazione ciclica, data da un certo numero di bit b. Potrò allora numerare $N = 2^b$ trame ma potrò averne in rete al massimo $N - 1$, altrimenti non potrei capire se il riscontro che viene ricevuto è quello della trama corretta oppure di quella con la stessa numerazione ma successiva. Per evitare di ricevere trame o riscontri sbagliati uso un **sistema a doppia finestra scorrevole**, con una finestra di trasmissione e una finestra di ricezione.

Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)



Le due finestre sono in realtà due **buffer**, rispettivamente di ampiezza W_s (trasmissione) e W_r (ricezione).
W_s rappresenta il numero di trame che il trasmettitore può inviare senza attendere i riscontri relativi, mentre W_r rappresenta il numero di trame che possono essere accettate dal ricevitore. Avendo un sistema di numerazione delle trame, anche gli "spazi" delle finestre risultano numerati, indicando quali trame possono essere trasmesse e accettate mediante l'estremo inferiore della finestra (rispettivamente L_s e L_r).

La trasmittente potrà inviare le trame con numero n che soddisfa la relazione

$$L_s \leq n \leq (L_s + W_s - 1)$$

La ricevente potrà accettare solo le trame con numero n di sequenza tale che

$$L_r \leq n \leq (L_r + W_r - 1)$$

Lo scorrimento in avanti della finestra avviene all'invio della trama corrispondente a L_s per la trasmittente e alla ricezione della trama corrispondente a L_r per la ricevente.

In reti molto grandi e/o complesse il controllo errori e controllo di flusso può essere fatto sia con FEC che con ARQ: vengono in ogni caso eseguiti entrambi i controlli, perché è più probabile che si siano errori rispetto ad una rete semplice vista la mole di informazioni che circola. Potrebbe anche succedere che la quantità di informazioni inviate è superiore a quella trasmissibile dalla rete: per quello viene svolto anche un controllo di flusso oltre che di errore.

A seconda delle dimensioni delle finestre avrò prestazioni differenti: sicuramente dovrò avere finestre di trasmissione e di ricezione di dimensione minore di N (numero di trame, $N = 2^b$), ma al di sotto di una certa soglia non sfrutterò completamente il tempo a disposizione per trasmettere (istanti di idle in invio). Per evitarlo deve essere vero che

$$\boldsymbol{W_s T_f \geq T_f + T_a + 2\tau}$$

per evitare perdite di efficienza nella trasmissione. **Se vale l'uguaglianza trasmetto in continuazione**.

Nei protocolli Continuous ARQ esistono due tipi di riscontri:
- **ACK i**: riscontro positivo cumulativo, fino alla trama i la trasmissione è corretta
- **NACK i**: riscontro negativo, la trama i non è stata trasmessa correttamente

- *Continuous ARQ: Go-Back-n (GBn)*

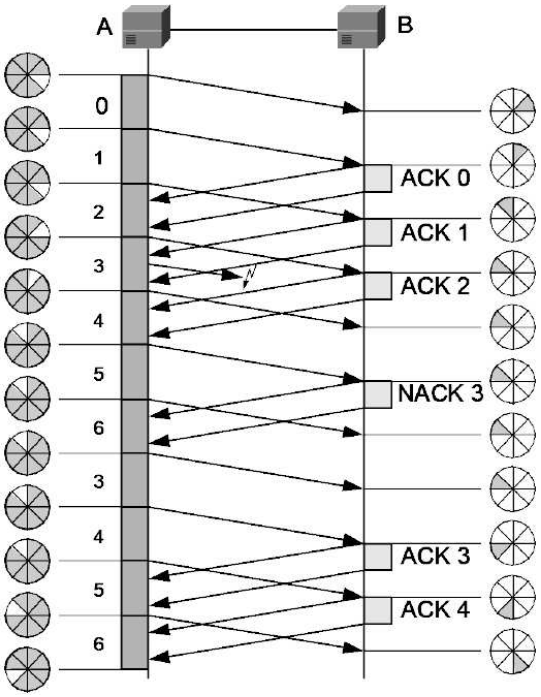
Il protocollo accetta solamente le trame che sono strettamente in sequenza, numerate ciclicamente con numerazione in base $N = 2^b$. In Go-Back-n tutte le trame ricevute fuori sequenza rispetto a quella corretta vengono scartate, viene inviato un riscontro negativo

Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

(NACK) e vengono ritrasmesse a partire dall'ultima trama non riscontrata (o riscontrata non correttamente).

Quando viene inviato il NACK i ha sia significato di riscontro negativo per la trama i, ma anche di riscontro positivo cumulativo fino alla trama i-1. Possono essere trasmesse, senza attesa di riscontro, fino a $n = N - 1$ trame.

In GBn la finestra di ricezione ha dimensione $W_r = 1$, perché può accettare solamente trame in sequenza, mentre la finestra di trasmissione avrà al più dimensione $W_s = N - 1$ al fine di evitare ambiguità con i riscontri.



Fintanto che ricevo ACK so che fino a quel punto la trasmissione è corretta: quando viene ricevuta una trama fuori sequenza la finestra di ricezione viene chiusa, scartando tutto, e viene inviato un NACK i. Viene confermato che la trasmissione è corretta fino alla trama i-1, ma viene chiesta la ritrasmissione da i in poi.

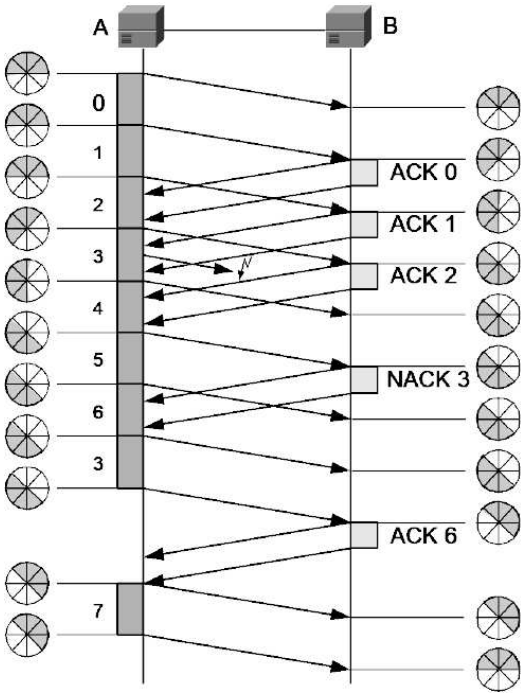
- *Continuous ARQ: Selective Repeat (SR)*
  In Selective Repeat posso accettare anche la ricezione di trame fuori sequenza, ma è necessario avere un buffer per poterle mettere nella sequenza corretta. Quando viene inviato un **NACK i però indica solamente il riscontro negativo per la trama i**, non da nessuna informazione sulle trame precedenti.

  In SR la ritrasmissione di una trama può avvenire per motivi diversi: la ritrasmissione selettiva può avvenire con una richiesta esplicita (invio e ricezione di un NACK) oppure per scadenza del timeout.

  In SR la dimensione della finestra di ricezione non sarà più unitaria: condizione necessaria per non creare ambiguità è che $W_s + W_r \leq N$, con scelta ottimale pari a $\boldsymbol{W_s = W_r = \frac{N}{2}}$.

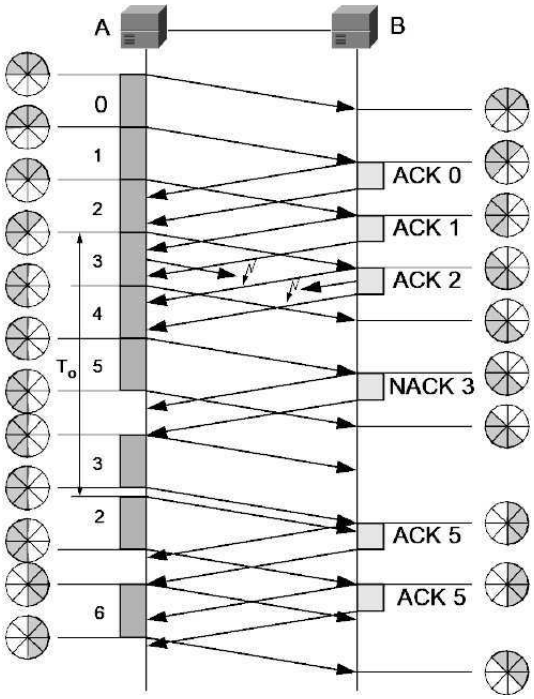
Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

Mano a mano che i pacchetti vengono ricevuti, vengono mantenuti nel buffer fino a che la finestra di ricezione non viene fatta ruotare. La finestra ruota solamente quando viene inviato l'ACK della trama corrispondente all'estremo inferiore della finestra di ricezione.



Quando una trama non viene ricevuta correttamente viene inviato un NACK: mentre questo viene trasmesso alla stazione sorgente, la ricevente continua ad accettare le trame che fanno parte della finestra di ricezione, anche se "rompono" la sequenza corretta. Viene segnalato alla sorgente di ritrasmettere la trama corrotta e, quando arriverà a destinazione correttamente, allora la finestra di ricezione potrà ruotare.

La ricevente invia il NACK quando riceve una trama fuori sequenza: se la trama arriva "in ritardo" e viene nuovamente inviata dalla sorgente, la nuova trama viene scartata.



Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

Il timeout relativo ad una trama scade quando non viene ricevuto dalla sorgente l'ACK relativo alla trama stessa: alla scadenza la trama viene inviata di nuovo. È possibile che sia andato perso solamente l'ACK della trama, che in realtà è arrivata correttamente a destinazione: la trama viene reinviata ma, se è stata ricevuta correttamente non viene accettata perché il suo "slot" nella finestra di ricezione non sarà disponibile.

Il funzionamento del timeout è uguale sia in GBn che in SR.

Sia in GBn che in SR, se vale $W_s T_f = T_f + T_a + 2\tau$ (non strozzo il canale e ho trasmissione continua), l'efficienza del protocollo sarà $\eta = 1$. Nel caso in cui non sia vera la precedente, calcolo l'efficienza come

$$\eta = \frac{W_s T_f}{T_f + 2\tau}$$

Posso calcolare l'efficienza anche come $\eta = \frac{1}{N_s}$ con N_s numero medio di ritrasmissioni che effettuo. Posso calcolare il numero medio di ritrasmissioni, data P probabilità trama errata e K numero di trame ritrasmesse in seguito ad un errore, come:

- Go-Back-n

$$N_s = 1 * (1 - P) + (K + 1) * P * (1 - P) + \cdots + (iK + 1) * P^i * (1 - P) \ldots$$

$$\Rightarrow \; N_s = \sum_{i=0}^{\infty} (iK + 1) * P^i * (1 - P)$$

$$\Rightarrow \; KT_f \cong 2T_f + 2\tau \quad \rightarrow \quad K = 2 + 2a$$

$$\Rightarrow \; \boldsymbol{N_s = \frac{1 + P + 2aP}{1 - P}}$$

dove $a = \frac{\tau}{T_f}$ tempo di propagazione normalizzato.

- Selective Repeat

$$N_s = \frac{1}{1 - P}$$

Nel SR l'efficienza è indipendente dalla lunghezza del percorso, ma dipende solamente dalla probabilità di errore. In GBn invece entra in gioco anche il tempo di propagazione normalizzato (e quindi tempo di propagazione e tempo di trasmissione). Nel caso in cui una finestra "strozzi" devo sostituire W_s a K, considerando il numero effettivo di pacchetti persi per la strozzatura.

- *Scegliere tra GBn e SR*

In casi in cui la probabilità di errore è bassa GBn è più conveniente di SR: l'efficienza di quest'ultimo inizia a calare quando si strozza il canale. GBn conviene in casi in cui la probabilità di errore è bassa perché ha una finestra di trasmissione più ampia rispetto a SR. In casi in cui è più probabile incorrere in errore solitamente è più efficiente SR, in particolar modo se la finestra di trasmissione di GBn è molto grande: verrebbero scartati molti pacchetti, facendo diminuire rapidamente l'efficienza.

Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

Protocollo HDLC (High level Data Link Control)

In HDLC vengono definiti 3 tipi di stazione:

- **Primaria**: responsabile del controllo del collegamento, emette trame dette comandi
- **Secondaria**: opera sotto il controllo di una primaria, emette trame dette risposte
- **Combinata**: combinazione delle precedenti, emette sia comandi che risposte

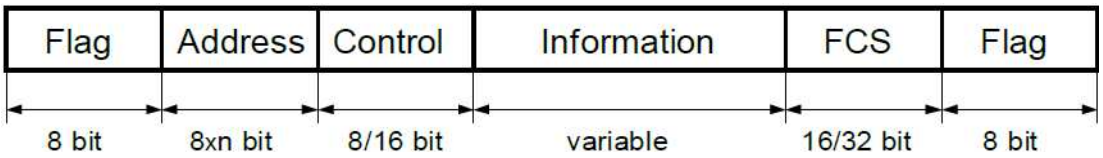
A seconda del tipo di stazioni collegate tra loro si distinguono due configurazioni di collegamento:

- **Sbilanciata**: 1 primaria, una o più secondarie
- **Bilanciata**: due stazioni combinate

Si distinguono tre diversi modi di trasferimento:

- **ABM** (Asynchronous Balanced Mode): configurazione bilanciata, 2 stazioni combinate con trasmissione di tipo full duplex
- **NRM** (Normal Response Mode): configurazione sbilanciata, 1 primaria e almeno 1 secondaria, trasmissione di tipo half duplex, la secondaria risponde solo ad un comando della primaria
- **ARM** (Asynchronous Response Mode): configurazione sbilanciata, come NRM ma il secondario può trasmettere senza il permesso, senza attendere un comando

- *Formato di trama*
  In una trama del protocollo HDLC si distinguono alcuni campi con funzioni diverse:



  - **Flag**: campo di 8 bit presente in testa e in coda alla trama, in HDLC ha sempre valore 01111110 e ha funzione di delimitazione della trama stessa. Se all'interno del messaggio sono presenti cinque 1 consecutivi opero bit stuffing, ovvero aggiungo uno 0 per evitare di avere sei 1 consecutivi e generare ambiguità o errori. Questo 0 di stuffing viene rimosso a destinazione ancora prima di interpretare la trama, con un'operazione di allineamento

  - **Address**: campo di lunghezza multipla di 8 bit, specifica la stazione secondaria ricevente in caso di comandi oppure la trasmittente in caso di risposte. Se composta da soli 1 indica che la trama viene inviata da una primaria a tutte le secondarie (con 8 bit massimo 128 secondarie)

  - **Control**: lunghezza 8-16 bit, specifica il tipo di trama in trasmissione, che può essere information, supervisory oppure unnumbered.

    - *Information (I)*: contiene informazioni per il livello superiore. I campi presenti sono:
      - N(S): contiene il numero della trama corrente
      - N(R): contiene il numero della prossima trama attesa, ha funzione di feedback fino alla trama corrente (ACK e NACK)

Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

- *Supervisory (S)*: primi due bit posti a 10, viene usata solo per riscontri o per segnalare che la stazione è fuori servizio. Riscontra le trame precedenti solo se previsto dal protocollo ARQ
    - Receive Ready (RR): comunica ACK quando non vengono inviate trame informative e segnala la prossima trama informativa attesa con N(R)
    - Receive Not Ready (RNR): conferma ricezione fino a N(R)-1 e segnala l'impossibilità a ricevere trame informative
    - Reject (REJ): comunica NACK per fuori sequenza con GBn, indicando che N(R) è mancante e riscontra correttamente fino a N(R)-1
    - Selective Reject (SREJ): comunica NACK per fuori sequenza con SR, richiedendo la ritrasmissione della trama N(R)

- *Unnumbered (U)*: primi due bit posti a 11, svolge funzioni di gestione del collegamento dati, ha funzione di segnalazione

- **Information**: lunghezza variabile ma multipla di 8 bit, può essere assente

- **FCS (Frame Check Sequence)**: lunghezza 16 o 32 bit, svolge funzione di rilevazione errori

Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

LAN (Local Area Network)
Una LAN è un sistema fisico di comunicazione tra apparecchiature indipendenti (stazioni) in un'area geograficamente limitata, ad alta velocità e con tasso d'errore generalmente basso. Dispositivi e apparecchiature di rete sono semplici e consentono la realizzazione di reti economiche, oltre ad essere affidabili e flessibili: non hanno particolari garanzie sulle prestazioni, a causa della semplicità delle tecnologie utilizzate, ma tutti gli apparecchi sono prodotti seguendo degli standard, rendendoli intercambiabili.

La LAN è una rete facilmente espandibile, sia per via della sua dimensione limitata sia grazie alla semplicità della rete: a differenza della WAN in caso di modifiche non è necessario ridefinire la topologia della rete. Quando viene apportata una modifica (es. aumento della velocità) vengono modificate solo le apparecchiature facenti parte della LAN stessa, sfruttando al massimo l'infrastruttura preesistente.

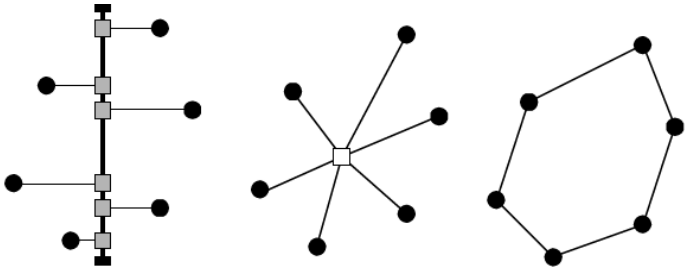
- *Protocolli di accesso rete LAN*
  Le regole che le diverse stazioni devono seguire per poter comunicare sulla rete viene detto protocollo di accesso: i protocolli della rete LAN vengono detti tecniche di accesso multiplo, poiché più apparecchi accedono allo stesso mezzo trasmissivo in un'area locale.

- **Tecniche di canalizzazione**: canalizzo (suddivido) una parte di qualcosa (banda, tempo, ecc.) assegnandola alle stazioni, a prescindere dall'utilizzo effettivo del mezzo trasmissivo da parte degli utilizzatori.
  - *FDMA* (Frequency Division Multiple Access): ogni stazione accede al mezzo trasmissivo usando una porzione di banda preassegnata
  - *TDMA* (Time Division Multiple Access): ogni stazione accede a tutta la banda disponibile ma in un intervallo di tempo assegnato
  - *CDMA* (Code Division Multiple Access): tutte le stazioni accedono a tutta la banda disponibile contemporaneamente, si distinguono le trasmissioni con il codice usato per generare il segnale dalle diverse stazioni

- **Accesso dinamico**: assegno l'accesso alla rete in modo dinamico e non arbitrario. Esistono tecniche centralizzate, in cui una stazione base regola l'accesso al mezzo trasmissivo e sono presenti due canali di comunicazione (uno con la stazione base per regolare l'accesso, l'altro bidirezionale per comunicare), e tecniche distribuite, in cui l'accesso viene gestito dalle diverse stazioni ed è presente un solo canale di comunicazione (con una parte riservata alla regolazione dell'accesso).

  - *Accesso ordinato*: tecnica distribuita, viene determinata una sequenza di accesso
    - Prenotazione: usata nelle reti con topologia a bus
    - Trasferimento di permesso: usata nelle reti con topologia ad anello
  - *Accesso a contesa*: tecnica ad accesso dinamico, detta anche ad accesso casuale perché le stazioni sono sempre libere di iniziare una trasmissione, potendo causare anche perdita di informazioni. È necessario risolvere le contese tra stazioni: Ethernet ricade qui dentro

- *Topologia di rete*



La struttura di connessione delle apparecchiature di comunicazione di una LAN determina solitamente le modalità di trasmissione dei dati: distinguiamo alcune topologie per la rete LAN.

Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

- **Topologia a bus**: tutte le stazioni sono collegate ad un unico mezzo trasmissivo che funge da bus, comprende anche trasmissioni bidirezionali (sia full duplex che half duplex). Le stazioni sono sia sorgente che destinazione delle trame che vengono trasmesse dalla rete.

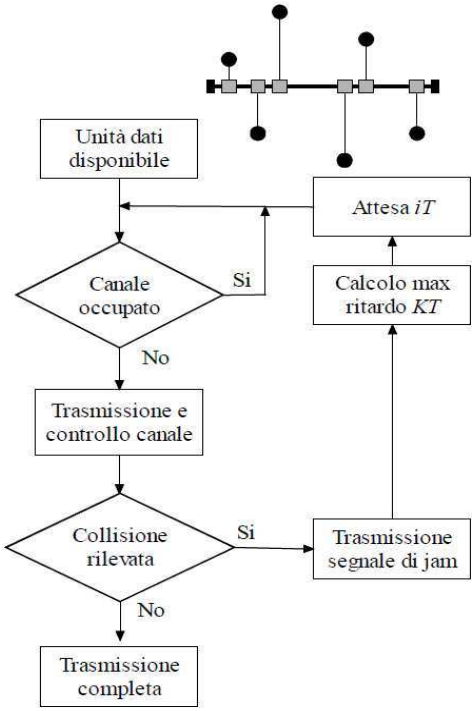
    - Accesso a prenotazione su rete a bus: N stazioni collegate con questa tecnica di accesso sfruttano il canale in modalità half duplex. Viene assunto che tutti gli utenti siano temporalmente sincronizzati: ad un certo istante inizia la fase di prenotazione, durante la quale ogni utente sceglie se comunicare o meno (1 per prenotare la comunicazione, 0 se non comunica).

      La prenotazione è broadcast, viene inviata a tutti gli utenti collegati, che sanno quanti utenti dovranno trasmettere: è possibile richiedere anche la quantità di dati che verrà trasferita o il tempo necessario per la trasmissione. Vengono effettuate le trasmissioni e poi si ripete ciclicamente. Non viene utilizzata.

    - Accesso a contesa su rete a bus: molto diffusa, tipicamente con metodo di accesso CSMA/CD (Carrier Sense Multiple Access with Collision Detection). Fare "carrier sensing" significa sondare il canale per vedere se qualcuno sta trasmettendo in quel momento: se non vengono rilevati segnali la stazione inizia a trasmettere.

      Può accadere che due stazioni facciano carrier sensing nello stesso istante, iniziando la comunicazione: nascono collisioni che rendono le trame inutilizzabili. Viene effettuata anche "collision detection": durante la trasmissione il segnale sul bus viene monitorato per controllare di avere un solo segnale e non generare collisioni tra trame.

      Quando viene rilevata una collisione la trasmissione viene interrotta, in modo da poter essere ripresa più tardi: quando due utenti rilevano una collisione bloccano la propria trasmissione per tempi diversi, in modo da riuscire statisticamente ad evitare nuove collisioni. Quando avviene una collisione viene inviato broadcast un segnale di jam, per comunicare l'avvenuta collisione a tutti.



Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

Per schedulare la ritrasmissione viene utilizzato un algoritmo di binary exponential backoff:

- T= unità temporale base (trasmissione minima, $51{,}2\mu s$)
- iT=tempo di attesa
- i=variabile casuale tra $[0, K]$
- $K = 2^k - 1$ con $k = \min(n, 10)$
- n=numero di tentativi di ritrasmissione già effettuati (numero di collisioni). Se arrivo a $n = 16$ comunico a IP che la rete non funziona (standard Ethernet).

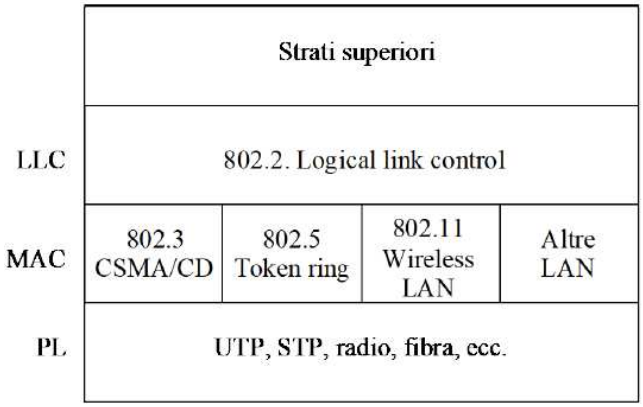
- **Topologia a stella**: è la topologia più utilizzata in reti locali, sia wireless che cablate. È necessario avere un hub (nel caso di reti cablate) o una wireless base station (nel caso di reti wireless) che rendano possibile il collegamento tra tutti gli utenti

- **Topologia ad anello**: stazioni connesse a due a due formando un anello. Tipicamente la trasmissione è unidirezionale e la trasmissione di una trama passa attraverso tutte le stazioni prima di quella di destinazione. Esiste una tipologia applicabile (Token-Ring), ma ha scarsa applicazione (se un link non funziona non funziona correttamente la rete)

- *Standardizzazione IEEE 802*
  Gli standard LAN vengono definiti dalla IEEE (Institute for Electrical and Electronics Engineers) con il nome di progetto IEEE 802. Lo standard IEEE 802 consiste in una serie di "raccomandazioni": stabilisce come deve essere strutturato il livello 2, come deve comunicare con i livelli 2 di altri host e come deve comunicare con gli altri livelli.
  Ad oggi in fase di standardizzazione troviamo la connessione a 400Gbit/s.

  Nello standard IEEE 802 vengono definiti 3 strati, al di sotto degli strati superiori che formano la rete:

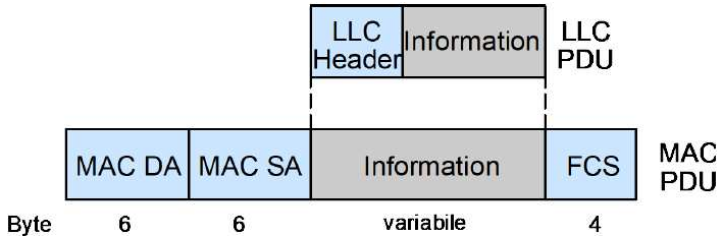
  - **LLC (Logical Link Control):** gestione collegamenti logici di livello 2. Non è più utile, potrebbe essere utilizzato per identificare connessioni connectionless (con o senza riscontro) o connection-oriented (con richiesta di permesso e attesa di riscontro)
  - **MAC (Medium Access Control):** gestione accesso al mezzo trasmissivo, condiviso nelle LAN
  - **PL (Physical Layer):** interfacciamento della stazione con il mezzo trasmissivo, con compiti di codifica/decodifica dei bit trasmessi/ricevuti



IEEE 802

Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

- *MAC (Medium Access Control)*
  Lo strato MAC provvede alla consegna delle unità informative tra stazioni di origine e destinazione: posso implementare un algoritmo di accesso diverso a seconda della tecnologia, ma è ricorrente assegnare un MAC Address ad ogni dispositivo per identificarlo al livello 2. Il livello MAC risolve il problema della condivisione del mezzo trasmissivo nelle reti LAN: le comunicazioni vengono trasformate da broadcast a punto-punto (ultimo bit 0), punto-gruppo (ultimo bit 1) o broadcast.



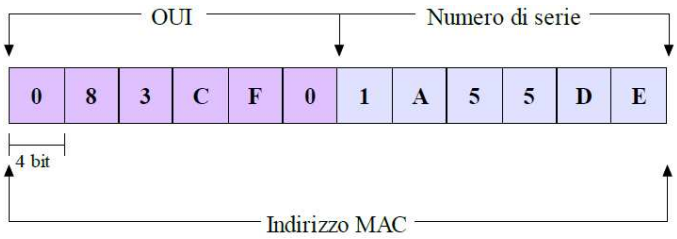
  Per il livello MAC è definita una MAC PDU (MAC Protocol Data Unit) come sopra, che presenta degli header del livello.

  - **MAC DA (MAC Destination Address)**: indica l'indirizzo MAC del destinatario dell'unità informativa
  - **MAC SA (MAC Source Address)**: indica l'indirizzo MAC del mittente dell'unità informativa
  - **Information**: contiene la PDU del livello LLC (se presente) e le informazioni
  - **FCS** (Frame Check Sequence): serve per verificare l'integrità dell'unità informativa

  Quando una trama viene ricevuta, viene subito verificata l'integrità (operazione matematica, se non integra viene scartata), viene analizzato l'indirizzo, verificando se la trama è destinata alla stazione stessa, e invia la trama al livello LLC se è destinata ad un gruppo di cui fa parte oppure è un broadcast.

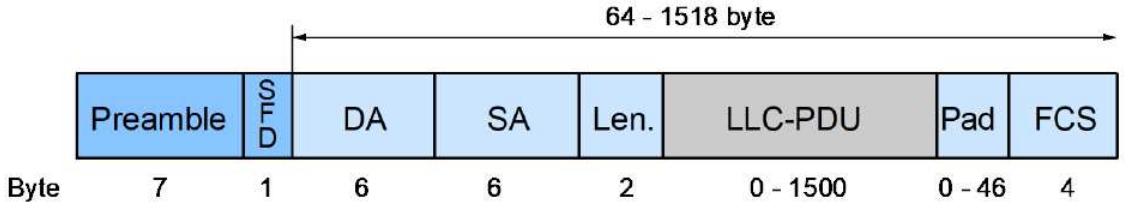
- *Struttura MAC Address*
  L'indirizzo MAC di un dispositivo è una sequenza di 12 simboli esadecimali: i primi 6 sono assegnati dalla IEEE al produttore della scheda (OUI, Organization Unique Identifier), i rimanenti sono assegnati dal produttore per distinguere ogni scheda da tutte le altre prodotte.

  Ogni simbolo esadecimale corrisponde a 4 bit, quindi il MAC Address risulta composto da 6 byte (48 bit): esistono 2^{48} indirizzi MAC.
  Gli ultimi due bit del primo byte del OUI hanno funzione di distinzione tra indirizzi locali e globali (penultimo bit, all'interno della rete o in tutte le reti) e di distinzione di indirizzi "singoli" o "di gruppo" (ultimo bit, unicast o multicast)



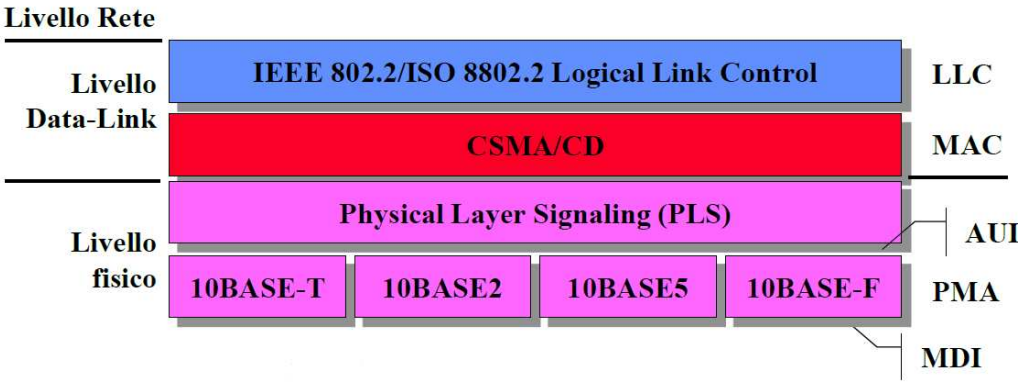
Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

- *Standard Ethernet*
  Lo standard Ethernet viene inventato nel 1976 in California, ma la prima standardizzazione è del 1985 (per connessioni a 10Mbps). Ha avuto successo perché è stato il primo standard a diffondersi a livello globale. Esiste, anche per Ethernet, un formato di trama standard che comprende la PDU vista precedentemente:



- **Preamble (preambolo)**: 7 byte, consente la sincronizzazione tra ricevitore e trasmettitore senza utilizzare hardware aggiuntivo
- **SFD (Start Frame Delimiter)**: 1 byte, indica l'inizio di una nuova trama sfruttando una violazione di una codifica dei bit
- **DA (Destination Address):** MAC Address del ricevitore
- **SA (Source Address):** MAC Address del trasmettitore
- **Len (Length):** 2 byte, indica la lunghezza del campo Information che va da 0 a 1500 byte
- **LLC-PDU:** vedi descrizione precedente. Se il campo delle informazioni è vuoto la trama corrisponde al solo header (18 byte)
- **Pad (Padding):** 0-46 byte, viene aggiunto per non avere trame troppo corte, poiché la trama minima deve essere di 64 byte. Se LLC-PDU è almeno 46 byte allora non avrò padding, altrimenti avrò un valore da 1 e 46 dato da 46-(LLC-PDU)
- **FCS (Frame Check Sequence):** vedi precedente

- *Tipologie di rete Ethernet*



- **10Base-T:** stazioni connesse a stella ad un hub, ciascuna con due doppini UTP. I cavi sono di lunghezza massima 100 metri e la rete viene costruita con hub a cascata (topologia a doppia stella). Viene utilizzato il connettore RJ45.
- **10Base2:** viene utilizzato un cavo coassiale Thin (RG58), lunghezza massima del segmento 185m. Per collegare un nuovo utente è necessario tagliare fisicamente il cavo e aggiungere la presa.
- **10Base5:** viene utilizzato un cavo coassiale Thick (RG213), lunghezza massima segmento 500m ma al massimo 100 utenti per segmento (tutti a non più di 2,5m di distanza dall'altro)
- **10Base-F:** usa mezzi trasmissivi in fibra, non viene utilizzato anche se è possibile implementare soluzioni completamente ottiche

Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

- *Hub (o repeater)*
  Componente che lavora sul livello 1 (fisico), nella pratica funziona come un ripetitore multiporta. Con gli hub posso collegare più reti tra loro ed estendere la rete oltre i 5km (limite di Ethernet).

  Esistono delle regole per le reti Ethernet multi-segmento:
  - **Mixing segment:** connette più di 2 transceiver (connessione a bus)
    - *10Base5*: cavo coassiale thick
    - *10Base2*: cavo coassiale thin
    - *10Base-FP*: fibra ottica passiva
  - **Link segment:** connette solo 2 transceiver, connessione punto-punto
    - *10Base-T*: doppino UTP
    - *10Base-FL*: fibra ottica
    - *10Base-FB*

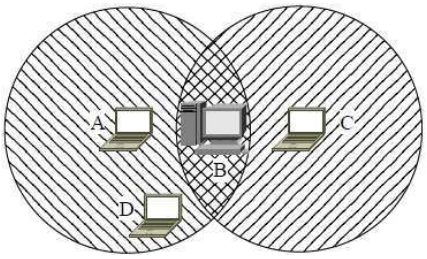
- *Standard velocità Ethernet*
  Viene chiamata **Fast Ethernet** la connessione almeno a 100Mbit/s, 10 volte più veloce di quella normale. Le reti Fast Ethernet non vengono realizzate con hub ma con switch, che sono in grado di leggere il pacchetto che arriva e indirizzarlo correttamente grazie alla forwarding table. In generale con uno switch due pacchetti non possono collidere (a patto di avere un buffer), mentre l'hub invia sempre tutti i pacchetti su tutti le linee (lo switch invia in broadcast solo se richiesto).

  Fast Ethernet mantiene il MAC di Ethernet standard con CSMA/CD. Mantiene anche il formato della trama ma cambiano i parametri fisici: ha velocità di 100Mbit/s e un tempo di bit di 10ns. Le reti Fast Ethernet possono essere sia full-duplex che half-duplex (coprendo distanze molto inferiori nel secondo caso).

  Esiste anche **Gigabit Ethernet**: mantiene CSMA/CD, è altamente scalabile e permette di velocizzare la LAN Ethernet con la sostituzione degli apparati di rete, contenendo i costi. Fornisce una banda di 1Gbit/s ed è possibile sfruttarle sia in full-duplex (connessioni switch-switch o switch-end) sia half-duplex (con l'uso di hub, cambia il formato della trama rispetto a quella standard). Viene principalmente usato come backbone per collegare gli switch tra loro. È possibile alzare ulteriormente la velocità: si sale allo standard 10Gbit/s, con connessioni solo full-duplex.

- *Wireless LAN*
  Si possono considerare reti realizzate con collegamenti senza fili invece che con collegamenti fisici: viene utilizzata una certa banda per realizzarle (tipicamente 2.4GHz o 5GHz) e possono avere diversi metodi di accesso. Lo standard che regola le connessioni Wi-Fi è IEEE 802.11.

  Si distinguono due tipologie di rete:
  - Basic Service Set (BSS): stazioni collegate tra loro che comunicano senza restrizioni, collegamento detto anche ad-hoc
  - Extended Service Set (ESS): deve essere presente un'ulteriore stazione che collega tra loro le stazioni e permette la comunicazione remota, detta Access Point (AP). Più BSS, ciascuno dotato di AP che li mette in comunicazione, formano ESS.
    Gli AP sono collegati tra loro tramite un sistema di distribuzione (DS, Distribution System), solitamente una LAN cablata.



Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

Nelle reti Wireless non viene utilizzato CSMA/CD, perché è impossibile rilevare le collisioni in un mezzo trasmissivo non univocamente determinato, ma dipendente dalla posizione. Esiste il problema della "stazione nascosta.

La stazione A sta trasmettendo a B: C non può rilevare questa comunicazione e inizia a trasmettere verso B, determinando una collisione. Per questo nelle Wireless LAN viene utilizzato **CSMA/CA (CSMA con Collision Avoidance)**: viene effettuato un sensing diverso rispetto a CD, con l'invio preventivo di una trama che chiede l'autorizzazione a comunicare (RTS, Request To Send), che determina l'invio di un riscontro (CTS, Clear To Send) nel caso sia possibile stabilire la comunicazione. Viene richiesto il riscontro esplicito delle trame.

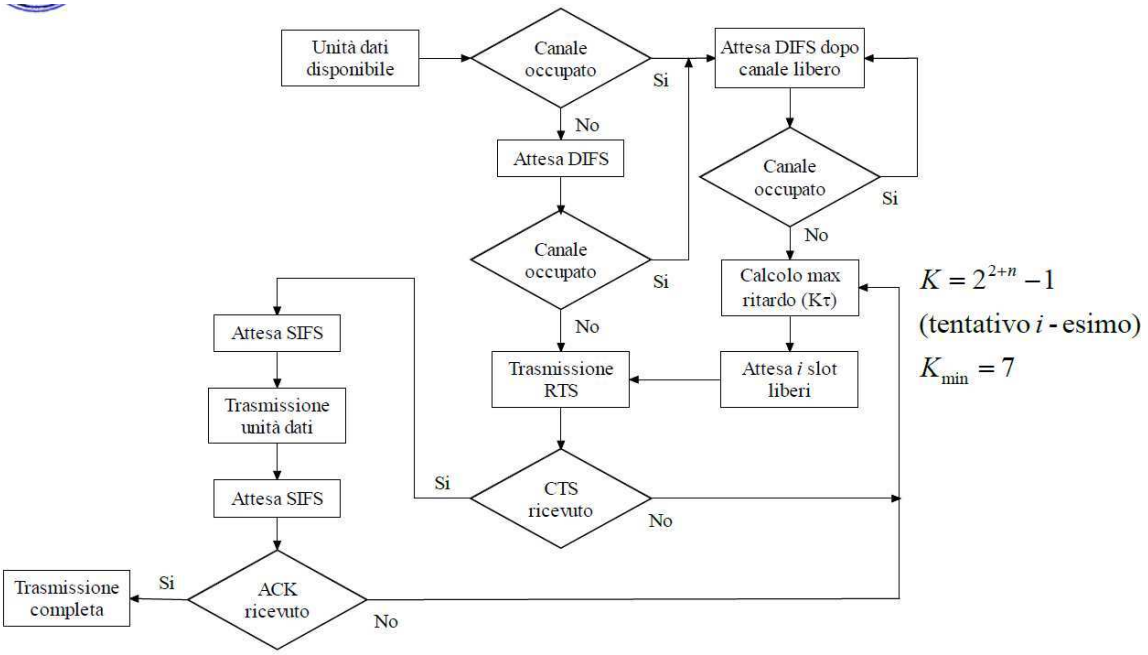
Esistono due modalità di accesso al canale radio:

- **Distibuted Coordination Funcion (DCF)**: può dare luogo a collisioni, sempre disponibile nelle stazioni della rete, adotta CSMA/CA
- **Point Coordination Function (PCF)**: procedura aggiuntiva opzionale, coordina l'accesso al mezzo trasmissivo evitando le collisioni

Per determinare le priorità di accesso al mezzo trasmissivo lo standard IEEE 802.11 utilizza una differenziazione dei tempi di ritardo di accesso al mezzo stesso.
Vengono definiti degli intervalli minimi di tempo tra trasmissioni di trame consecutive (IFS, Interframe Spacing):

- **SIFS (Short IFS):** il più breve, tempo di attesa prima dell'invio di trame ad alta priorità come CTS o ACK
- **PIFS (PCF IFS):** tempo intermedio, utilizzato per garantire priorità di accesso al mezzo in modalità PCF
- **DIFS (DCF IFS):** tempo più lungo, usato dalle stazioni DCF per trasmettere il primo frammento dati o di gestione

Il protocollo CSMA/CA evita le collisioni seguendo il procedimento seguente:



Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

- *Protocolli ad accesso casuale*
  Si possono distinguere alcune tipologie di protocolli ad accesso casuale:

  - **Protocolli con rivelazione del canale**
    - *Con rilevazione delle collisioni*
      - CSMA/CD: protocollo basato sul concetto di carrier sensing (rilevazione del canale), consente la trasmissione solamente quando il canale è libero e controlla lo stato del canale durante la trasmissione, interrompendola quando rileva delle collisioni
      - Slotted CSMA/CD

    - *Senza rilevazione delle collisioni*
      - CSMA: protocolli basati su carrier sensing, trasmettono solo con canale libero. Quando una stazione trova il canale occupato può agire in modi diversi:
        - Non-persistent: la trasmissione viene riprogrammata senza rimanere in ascolto sul canale
        - 1-persistent: continua ad ascoltare il canale fino a che non si libera, probabilità di trasmissione pari a 1
        - P-persistent: continua ad ascoltare il canale fino a che non si libera, quindi trasmet
        - te con probabilità p. Se ricade in $1 - p$ si comporta secondo logica non-persistent
      - Slotted CSMA
      - CSMA/CA: protocollo basato sul concetto di carrier sensing (rilevazione del canale), consente la trasmissione solamente quando il canale è libero, previene le collisioni applicando un protocollo di richiesta o autorizzazione esplicita per l'accesso

  - **Protocolli senza rilevazione del canale**
    - ALOHA: sono protocolli che non effettuano nessuna rilevazione, ogni stazione è libera di trasmettere in ogni momento
    - Slotted ALOHA

- *Protocollo ALOHA*
  Nei protocolli ALOHA ogni stazione è libera di trasmettere in ogni istante: il successo della trasmissione è dato solo dalla mancata collisione con altre trasmissioni. L'informazione inviata, quando viene ricevuta, viene riscontrata dal destinatario (invia un ACK alla sorgente): viene definito un tempo di attesa per il riscontro (timeout), solitamente pari a 2τ con τ tempo di propagazione tra le due stazioni più lontane. Quando viene violato il timeout viene rilevata la collisione tra trasmissioni.

  Per evitare la ripetizione di una collisione è necessario definire un algoritmo di rescheduling casuale delle informazioni (algoritmo di backoff): viene selezionato un intervallo di ritardo massimo KT, multiplo del tempo di trasmissione T di una qualunque trama, e si ritrasmette dopo un certo tempo iT dalla collisione, con $i \in [0; K - 1]$ scelto casualmente per ogni trama. Così facendo si distribuisce il carico e si riduce il rischio di collisioni.

- *Protocollo Slotted ALOHA*
  Slotted ALOHA funziona in modo simile ad ALOHA: l'orizzonte temporale viene suddiviso in slot temporali di durata $T = \frac{L}{C}$, con T tempo di trasmissione di un pacchetto. Tutte le stazioni sono sincronizzate tra loro: se trasmettessero contemporaneamente si sovrapporrebbero. Quando una stazione riceve un'informazione da inviare deve necessariamente aspettare l'inizio dello slot temporale successivo, non può inviare a metà: le trasmissioni si sovrappongono per intero oppure non accade, può succedere solo con gli ACK.

Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

In questo caso il timeout viene calcolato come $2\tau + \epsilon$, con ϵ adatto, perché 2τ potrebbe non essere multiplo di T. È complesso come sistema perché è necessario avere sincronia perfetta tra le diverse stazioni, oppure avere un tempo di guardia sufficiente ad assorbire le differenze di clock tra le stazioni. Il periodo di vulnerabilità del protocollo è pari a T. Ogni stazione ha probabilità di trasmettere in un dato slot pari a p: avrà probabilità di non trasmettere pari a 1-p.

- *Analisi efficienza protocolli ALOHA*
  Considero uno scenario con N stazioni, ciascuna con probabilità p di trasmettere in uno slot: la probabilità che non trasmetta sarà quindi $1 - p$. Se una stazione trasmette, la probabilità di successo della trasmissione è data dalla probabilità che tutte le altre $N - 1$ stazioni non trasmettano in quello slot. Vale quindi

$$P_s = (1 - p)^{N-1}$$

con P_s probabilità di successo. Segue quindi che la probabilità che in uno slot arbitrario una particolare stazione trasmetta è data dalla composizione delle due precedenti, ovvero

$$P_s = p(1 - p)^{N-1}$$

Conoscendo il numero N delle stazioni, la probabilità che qualunque stazione trasmetta con successo sarà

$$S = N * p(1 - p)^{N-1}$$

S è anche il numero medio di trasmissioni con successo in un singolo slot, che viene detto throughput. Possiamo definire il numero medio di trasmissioni sul canale, ovvero il traffico G, come $G = Np$: sostituendo nella formula del throughput la precedente vale che

$$S = Np(1 - p)^{N-1} = G\left(1 - \frac{G}{N}\right)^{N-1}$$

La formula precedente rappresenta l'efficienza di Slotted ALOHA, poiché dà il numero medio di successi in funzione del numero medio di trasmissioni effettuate.

Portando al limite la precedente, per $N \to \infty$, si ha

$$S = Ge^{-G}$$

Si ottiene quindi la massima efficienza per $G = 1$, pari a $S = \frac{1}{e} \approx 0{,}37$.

L'analisi per ALOHA standard è simile: sono possibili collisioni con sovrapposizione parziale delle comunicazioni, in un periodo di vulnerabilità $2T$. Dalle precedenti segue allora che

$$S = Np(1 - p)^{2(N-1)}$$

$$S = G\left(1 - \frac{G}{N}\right)^{2(N-1)}$$

Per ALOHA, con $N \to \infty$, vale che $S = Ge^{-2G}$: si ha efficienza massima per $G = 0{,}5$ e $S = \frac{1}{2e} \approx 0{,}18$. Teoricamente aumentando il traffico dovrebbe aumentare anche il throughput di conseguenza: nella realtà non è così, perché aumenterebbero le collisioni, si allungherebbero i tempi di occupazione dei buffer, ecc.
Non sfrutterei a pieno la capacità della rete.

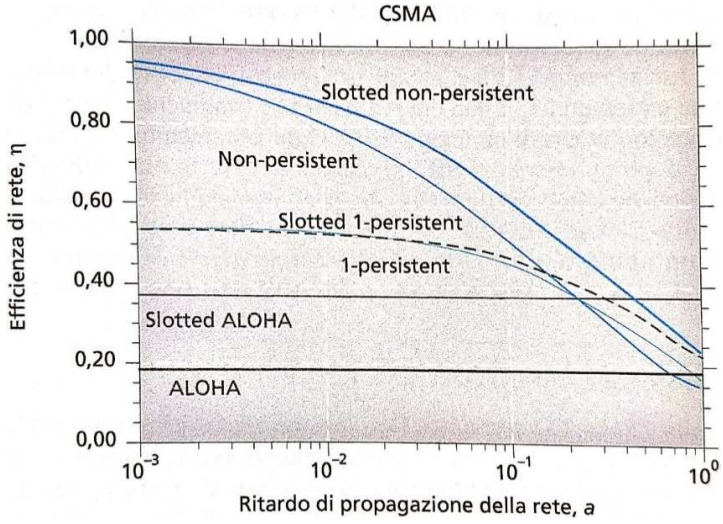
Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

- *Protocolli CSMA*
  I protocolli CSMA si distinguono da quelli ALOHA per la funzione di "carrier sensing": ascoltano il canale prima di iniziare a trasmettere, in modo da ridurre le collisioni. Anche con i protocolli CSMA è possibile suddividere il tempo in slot, sincronizzando le stazioni tra loro e facendo carrier sensing alla fine di ogni slot (non ci sono sovrapposizioni).

  Il carrier sensing può avvenire secondo diverse logiche:
  - **Non-persistent**: rilevo canale occupato, rischedulo la trasmissione ad un istante successivo
  - **1-persistent**: continuo ad ascoltare il canale finché non si libera, quindi trasmetto con probabilità 1
  - ***p*-persistent**: continuo ad ascoltare il canale finché non si libera, quindi trasmetto con probabilità p

  Il protocollo p-persistent ricade in non-persistent e 1-persistent nei casi limite, ovvero $p = 0$ e $p = 1$. I protocolli slotted prevedono il sensing solo all'inizio di un nuovo slot temporale, per non sovrapporre le trasmissioni, mentre quelli standard (tranne non-persistent) ascoltano il canale in continuazione, rilevando il primo istante libero per la trasmissione.

- *Analisi efficienza protocolli CSMA*



  I protocolli CSMA hanno efficienza generalmente maggiore rispetto ai protocolli ALOHA: cala vistosamente all'aumentare di $a = \tau/T$ ritardo normalizzato, fino a scendere anche al di sotto dei protocolli ALOHA. I protocolli CSMA sono poco efficienti nelle reti molto grandi, perché la procedura di sensing perde efficacia. Minore è il valore di a e maggiore è l'efficienza, dato che l'inefficienza dovuta alla propagazione diminuisce e il sensing diventa più efficiente.

- *CSMA/CD*
  È un'evoluzione di CSMA, permette di ascoltare il canale durante la trasmissione stessa: quando viene rilevata una collisione la trasmissione viene interrotta e viene inviato a tutte le stazioni un segnale di jam, che fa scartare la porzione di informazione che hanno memorizzato. Così facendo si riduce l'intervallo di collisione, ovvero il tempo perso per quelle circostanze. Anche per CSMA/CD sono definite le classi di protocollo standard, ovvero slotted, non-slotted, non-persistent, 1-persistent e p-persistent.

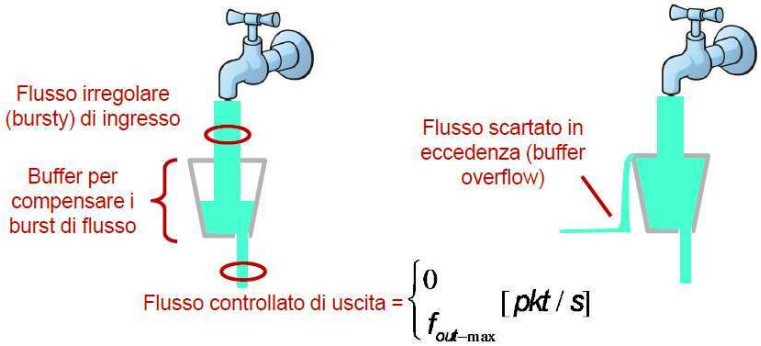
Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

Controllo di flusso e controllo di congestione
Con il controllo di flusso si rinuncia a sfruttare il 100% della capacità della rete, controllando però la trasmissione e agendo in favore della stabilità della stessa. In generale controllo di flusso e congestione sono cose diverse: il primo regola l'attività di una connessione in base allo stato della connessione stessa, il secondo la regola in base allo stato della rete da attraversare.

Classifichiamo le diverse strategie di controllo in base a:

- **Portata**: estensione geografica, può essere
  - *In accesso alla rete*
  - *Internodo*
    - Hop-by-hop
    - Edge-to-edge
  - *Da terminale a terminale* (end-to-end)
- **Tipologia**
  - Metodi reattivi: vengono attuati solo in caso di congestione, operano in anello chiuso
  - Metodi preventivi: vengono sempre attuati a prescindere dalla collisione
- **Tecnica di controllo**
  - Credit-based: viene rilasciato un numero limitato di "crediti" (come nel controllo a finestra)
  - Rate-based: vengono imposti limiti al throughput

- <u>*Controllo di flusso Rate-based*</u>
  Per noi solo tecniche a bucket (leaky bucket e token bucket). Sono tecniche usate principalmente a livello 3, sia in accesso sia nei nodi. Il concetto di leaky bucket è il seguente:



  Leaky bucket funziona come un secchio con un buco: in entrata mando quanto mi pare ma esce sempre la stessa quantità. In entrata posso avere un segnale bursty ma l'uscita sarà sempre regolare, perché viene compensata dal buffer di pacchetti posto in mezzo. Se continuo a riempire il buffer di capacità W, posso anche arrivare al limite: perdo pacchetti quando ne ho accumulati W. Devo dare al buffer permessi a frequenza costante, consumandone uno per ogni pacchetto inviato, per trasmettere a frequenza costante: per massimizzare la frequenza in uscita devo sempre avere almeno un pacchetto nel buffer.

  Assumendo che i pacchetti siano tutti di dimensione costante L_b, posso dire che la frequenza media massima di uscita (bit/s) sarà $A_{b\,MAX} = L_b * v \leq P_b$,con v frequenza di generazione dei permessi e P_b frequenza di trasmissione sulla linea in uscita. Se si svuota il buffer e viene generato un permesso, questo viene perso: per questo è necessario avere sempre almeno un pacchetto nel bucket, altrimenti non si sfrutterebbero al massimo i permessi.

- <u>*Token bucket*</u>
  È una tecnica più adatta a sorgenti bursty: in questo caso ho un buffer di permessi e non di pacchetti. I permessi sono generati con frequenza v e ogni pacchetto trasmesso genera un permesso: se non ci sono pacchetti da trasmettere il permesso non viene perso ma rimane nel buffer. Vengono persi permessi solo se si supera la capacità del buffer W. Anche in token bucket vale $A_{b\,MAX} = L_b * v \leq P_b$.

Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

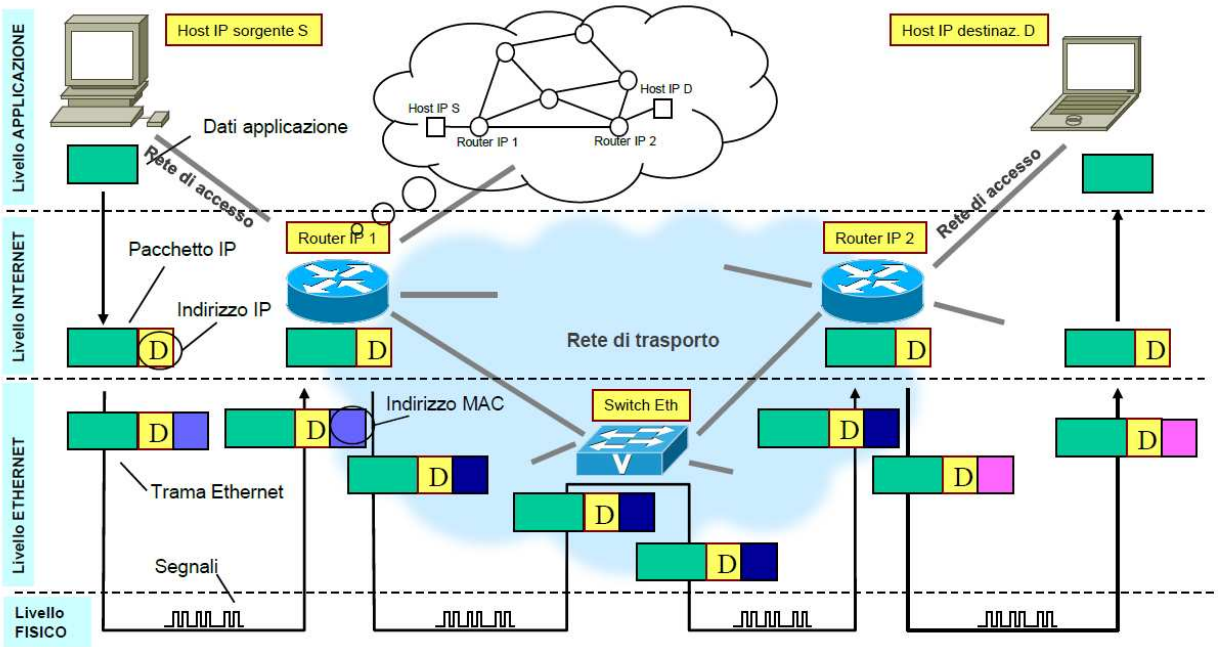
Internet
Parliamo del livello 3 della pila protocollare: il compito di Internet è quello di interconnettere tra loro più sistemi autonomi (reti di livello 2). Sono interconesse tra loro reti di tipo diverso, sviluppate indipendentemente l'una dall'altra (diversi apparati, protocolli e architetture). Viene così garantita la comunicazione end-to-end a qualunque coppia di host nel mondo, che hanno in comune la "lingua", cioè IP.

Il livello 3 non è presente negli switch (perché interconnette apparati con livello 2 in comune), ma solo in router e terminali: è il livello che connette tra loro le diverse reti, non è necessario risalire la pila ulteriormente se non si è giunti a destinazione, il pacchetto viene portato al livello applicativo solamente una volta raggiunto il terminale di destinazione.

Il livello 2 può essere differente in tutte le reti (WiFi, Ethernet, ecc.) ma tutte comunicano tra loro mediante IP, mascherando queste differenze e pemettendo la comunicazione.

- *Internet management*
  L'ente che si occupa del management di Internet è ISOC (Internet Society), inizialmente un ente indipendente di ricercatori e scienziati, che riunisce le società che presiedono all'organizzazione di Internet, ovvero:
  - **IAB (Internet Achitecture Board)**, coordina le attività di altri due gruppi importanti:
    - *IETF (Internet Engineering Task Force)*, che affronta le problematiche di sviluppo e realizza effettivamente i protocolli da implementare
    - *IRTF (Internet Research Task Force)*, che gestisce e sviluppa attività di ricerca

- *Rappresentazione rete Internet*
  Viene rappresentato il percorso svolto da un pacchetto nella rete.



La sorgente genera il pacchetto, che scende fino al livello 2 (Ethernet in figura) e viene incapsulato, per poi arrivare al router IP 1. Viene "cambiato" l'indirizzo MAC, mettendo quello della rete di destinazione, e dallo switch in poi non viene fatto nulla al di sopra del livello Ethernet, fino a giungere nella rete di destinazione ed al suo router IP. Il router 2 indirizza il pacchetto mediante l'indirizzo IP al dispositivo corretto, togliendo l'header precedente e mettendone uno "nuovo" che viene rimosso a destinazione, ottenendo il pacchetto.

Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

- *Architettura di Internet*
  La connettività viene garantita dagli ISP (Internet Service Provider, sono Telecom ecc.), che sono tra di loro collegati: la parte di rete gestita da un ISP viene detta autonomous system. Il livello 3 mette in comunicazione tra loro i diversi ISP, che si collegano tra loro negli exchange points. In base alle dimensioni della rete che gestiscono, gli ISP vengono suddivisi in 3 categorie:

  - **Tier-1 ISP**: ISP globali, gestiscono la backbone di Internet. Non acquistano capacità da nessuno ma la forniscono agli ISP di tier inferiori. Possono permettersi di trasportare i dati degli operatori inferiori senza farsi pagare, ma fanno trasportare i propri dati anche a questi operatori. Telecom è qui.
  - **Tier-2 ISP**: ISP che acquistano o noleggiano connettività dai Tier-1, fanno accordi di peering con essi in cui nessuno paga per veicolare i propri dati. Un esempio è Aemme a Milano.
  - **Tier-3 ISP**: solitamente ISP regionali, sono quelli che forniscono effettivamente l'accesso ad Internet agli utenti. Esempi possono essere un'università o una banca.

- *Strato di rete e trasferimento dei pacchetti*
  Lo strato di rete (livello 3) ha il compito di trasferire i dati tra gli host che ospitano i due processi applicativi comunicanti. Il percorso compiuto dai pacchetti non è noto a priori, ma viene deciso in modo distribuito (hop-by-hop) dai router che indirizzano i pacchetti di volta in volta. Ogni router dovrebbe conoscere tutti i router ai quali potrebbe inviare pacchetti, il che sarebbe complesso: con IP queste complessità vengono gestite, anche grazie a protocolli accessori che agiscono sempre al livello 3.

  L'obiettivo del livello 3 è quello di trasferire le unità informative (tra gli host che ospitano i processi comunicanti) provenienti dal livello superiore mascherando, ai livelli superiori stessi, i dettagli relativi al tipo di rete sottostante. Questo livello svolge quindi alcune funzioni fondamentali:
  - **Indirizzamento**: identifica univocamente l'interfaccia di rete di un host o di un router
  - **Instradamento**: determina il percorso da far compiere al pacchetto per raggiungere la destinazione
  - **Frammentazione**: divide i pacchetti molto grandi in molti pacchetti piccoli
  - **Multiplazione**: vengono multiplati più flussi di livello 3 su un singolo collegamento di livello 2
  - **Controllo di flusso e congestione**
  - **Intercooperazione di reti diverse tra loro**

- *Trasferimento di pacchetti tra host*
  I segmenti dello strato di trasporto vengono trasferiti dallo strato di rete dall'host sorgente all'host destinazione: tutti i protocolli dello strato di rete sono implementati anche in host e router. Lato sorgente i segmenti vengono incapsulati in datagrammi IP, che vengono inoltrati hop-by-hop fino all'host di destinazione: i router esaminano i campi dell'header di ogni datagramma che li attraversa, in modo da poterli instradare correttamente. A destinazione i segmenti vengono consegnati allo strato di trasporto.

  Insieme al protocollo principale dello strato di rete (protocollo IP), che viene usato per trasferire i dati dell'utente, vengono utilizzati anche dei protocolli accessori di segnalazione per supportare il trasferimento dati (ICMP, ARP, RARP, OSPF, ...).

- *Servizio datagramma*
  Il servizio datagramma non richiede una call di setup: i router non hanno cognizione dello stato della connessione end-to-end, non esiste il livello di connessione al livello di rete. I pacchetti vengono inoltrati utilizzando l'indirizzo di destinazione dell'host: pacchetti diversi con la stessa coppia sorgente-destinazione possono percorrere strade differenti.

Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

È la logica alla base di IP: l'instradamento è demandato ad ogni nodo nel quale viene generata una tabella di routing, con tutti i nodi di destinazione possibili e il passaggio successivo da fare per arrivarci (next hop).

Con questo servizio è possibile avere dei fuori sequenza, delle perdite di pacchetti o anche dei duplicati (a causa di un eventuale reinvio): IP è un protocollo "best effort", non ha garanzie di successo. La richiesta di ritrasmissione di eventuali pacchetti persi è compito di livelli superiori. IP è connectionless.

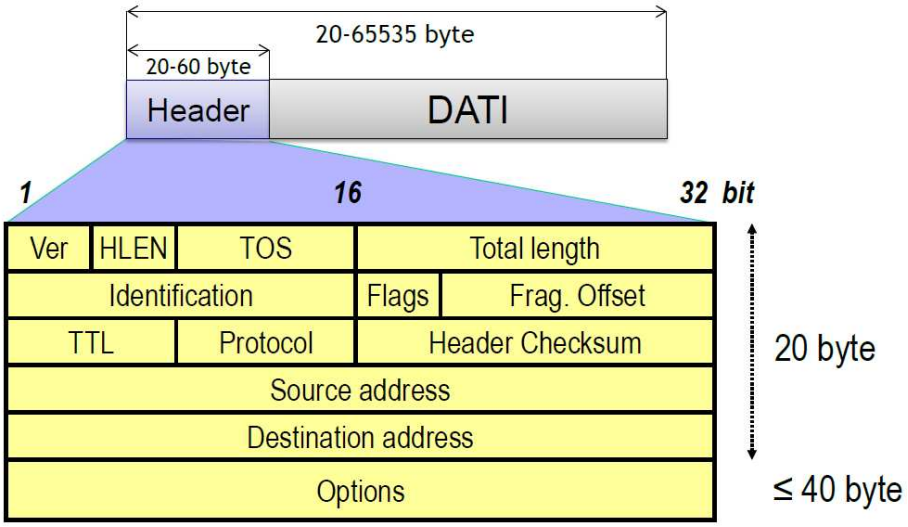
- *Servizio circuito virtuale*
  Le unità informative in questo servizio sono dette pacchetti: prima della fase di trasferimento dati avviene una fase di segnalazione, che serve per instaurare una connessione virtuale tra i due host che devono comunicare attraverso la rete. I pacchetti vengono trasferiti attraverso i nodi grazie ad una "etichetta di destinazione" (presente nell'header del pacchetto, contiene un ID di circuito virtuale) e ad una tabella di inoltro presente in ogni nodo.

  Il servizio di circuito virtuale si compone quindi di 3 fasi: apertura o instaurazione, trasferimento dati e rilascio. Nella fase di apertura viene inviato un pacchetto di richiesta di connessione che viene instradato, tracciando quindi un percorso end-to-end, detto circuito virtuale, costituito da una cascata di canali logici: queste informazioni vengono immagazzinate nella tabella di inoltro di ciascun nodo.

  In caso positivo viene trasmesso, in verso opposto, un pacchetto che indica alla sorgente l'apertura del circuito virtuale: inizia quindi la fase di trasferimento dati. La connessione virtuale termina con la fase di rilascio, che inizia su iniziativa di una delle due estremità e avviene mediante lo scambio di riscontri locali (messaggi generati tratta per tratta), in modo da cancellare il riferimento alla connessione dalla tabella di inoltro di ogni nodo coinvolto.

Protocollo IP

Le unità informative del protocollo IP sono chiamate datagrammi o pacchetti IP e sono formati da un header IP e dal payload (UI del livello di trasporto). Dal protocollo IP viene svolta la funzione di rivelazione errori sull'header e viene posta una garanzia sul max lifetime (dopo un certo tempo prestabilito il datagramma viene eliminato). L'header IP minimo è formato da 20 byte, organizzato come segue:



In linea di principio non è definita la lunghezza dell'header IP, perché possono essere presenti 40 byte opzionali (Options). Ad oggi vengono scartati direttamente dagli operatori perché venivano usati per fare attacchi hacker. L'header è composto da 5 linee (+1 di Options) da 32 bit.

Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

Sono presenti i seguenti campi:

- **Source address e destination address** (entrambi da 4byte o 32 bit): contengono rispettivamente l'indirizzo IP della sorgente e l'indirizzo della destinazione, indipendentemente dalla rete che percorre.
- **Campo Ver (version)** è da 4 bit, che assume solamente 2 valori: IPv4 o IPv6. Se il campo contiene informazioni diverse il pacchetto viene scartato.
- **Campo HLEN**: sempre 4 bit, specifica quanto è effettivamente lungo l'header in multipli di 32 bit (max 15, min 5).
- **Total length**: indica la lunghezza totale del pacchetto in byte (header+payload), sottraendo la lunghezza dell'header si ottiene quella del payload.
  La lunghezza massima del campo è 2^{16}-1 byte, mentre la lunghezza massima totale del pacchetto IP è 2^{16} byte.
- **TOS (Type of service)**: sono 8 bit. Un router supporta servizi diversi che hanno requisiti diversi. È utile avere una policy di utilizzo della coda (un pacchetto Skype deve passare prima di uno delle mail). Sarebbe bello, invece di fare FIFO (First In First Out), riuscire a gestire in questo modo i pacchetti. Nella realtà non sono molto utilizzati, anche se esistono, perché complicano la gestione della rete. All'interno di TOS ci sono due bit (ECN) che vengono usati come segnalazione (messi entrambi a 1) per far capire che tra poco potrebbero essere persi dei pacchetti.
- **Protocol**: specifica che protocollo è inserito sopra a IP (livello 4). In realtà esistono solo due protocolli, TCP e UDP. È costituito da 8 bit. Sopra al livello IP possono essere trasportati anche i protocolli accessori, che sono sempre di livello 3 ma che vengono trasportati mediante IP.
- **TTL (time to leave)**: viene settato ogni volta che il pacchetto IP parte. Quando parte dalla sorgente viene settato a 255 (sono 8 bit, vengono messi tutti a 1). Ogni volta che passa da un router (ad ogni "hop") viene sottratto 1 e, quando il campo TTL arriva a 0, il pacchetto viene scartato e viene generato un messaggio di errore verso la sorgente. Nella realtà anche per andare da una parte all'altra del mondo bastano 20-30 passaggi (vedi comando cmd traceroute).

La seconda riga della struttura sopra viene utilizzata per fare frammentazione: può succedere che, passando da una rete all'altra, sia necessario ridurre la lunghezza massima dei pacchetti. La massima dimensione del frammento viene chiamata MTU (Maximum Transmission Unit): tutto il pacchetto IP deve rimanere all'interno dell'MTU corrispondente (come dimensione) perché deve essere trasportato all'interno di una trama di livello 2.

MTU è diverso a seconda del livello 2 che consideriamo. Frammentare comporta un aumento degli overhead, perché deve essere messo un header a pacchetto (che non trasporta informazioni utili). I campi relativi alla frammentazione sono:

- **Fragment identification**: viene messo sempre nel pacchetto, anche se non viene frammentato. Ogni volta che invio un pacchetto, questo ha un valore di fragmented identification associato. La relazione tra questo e il source address/destination address ci dà informazioni su che frammento stiamo trasferendo. L'importante è che sia bassa la probabilità di trovare in rete nello stesso istante due pacchetti con lo stesso identification address. Ce ne sono 2^{16} diversi, sono 16 bit.
- **Fragment offset**: dice qual è il primo byte del frammento successivo, indicandolo in multipli di 8 byte. Nel primo pacchetto sarà nullo. È da 13 bit. Sono 2^{13} parole e ciascuna rappresenta 8 byte.
- **Flag**: 3 bit, il primo è sempre vuoto. Il secondo (bit D, do not fragment) indica quando un pacchetto non va frammentato per nessun motivo: se la rete ne richiede la frammentazione il pacchetto viene scartato. Il terzo (M, More fragments) è messo nullo (0=falso) solamente nell'ultimo frammento della sequenza.

Sono presenti anche altri campi all'interno dell'header IP, ciascuno con la sua funzione:

- **Header checksum**: codice a rivelazione d'errore sul solo header IP, non protegge il payload. Se viene rilevato un errore il pacchetto viene scartato e basta.

Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

- **Option**s: fino a 10 linee di 4 byte. Un esempio di options è il traceroute.
- **Padding**: campo che può non essere presente, viene utilizzato per rendere le righe multiple di 32 bit e rispettare lo standard.

Esempio 1 di frammentazione
I 20 byte che mancano tra payload e MTU sono quelli dell'header IP. Ho un pacchetto da 4000 byte di payload e 20 di header: arriva su una rete con MTU da 1420, mentre prima era 10000 (no problem). Il router prende il pacchetto e li divide con payload massimo 1400 (1420-header). Viene mantenuto nei frammenti il fragment identification: quando mi arrivano tutti i frammenti a destinazione so che appartengono tutti a uno stesso pacchetto. Il fragment offset è il numero del primo byte del frammento diviso 8 (ad es. 2800 corrisponde a 350).

Può anche accadere che un frammento venga frammentato a sua volta: non cambia niente, viene frammentato come se fosse un normale pacchetto. Se frammento nuovamente un pacchetto il primo frammento avrà fragment offset uguale a quello del frammento più grande originale. Quelli successivi vengono impostati sempre nello stesso modo, dividendo per 8 il numero del byte iniziale del frammento.

- *Indirizzamento IP*

Gli indirizzi IP sono assegnati globalmente dalla IANA (Internet Assigned Numbers Autority), che li assegna mediante sue divisioni (una per continente). Gli indirizzi IP sono da 32 bit (numeri da 0 a 255) e vengono convenzionalmente suddivisi in 4 byte. L'indirizzo minimo è 0.0.0.0 e il massimo 255.255.255.255. Un router ha tanti indirizzi IP tanti quanti le reti con cui si interfaccia, uno per rete: l'indirizzo IP, precisamente, è l'indirizzo di un'interfaccia, non di un host.

L'indirizzo IP viene diviso in due campi di dimensione variabile:
- NetID: bit necessari per identificare in che rete nel mondo mi trovo, identifica in che rete di che ISP sono.
- HostID: dice che utente sono all'interno della rete stessa.

Non posso usare tutti i numeri che voglio, ad esempio quello con tutti 0 (quello con HostID 0.0) è l'indirizzo che viene utilizzato per identificare l'intera rete. Non mando mai un pacchetto alla rete ma sempre ad un host, ma se non avessi un indirizzo della rete dovrei avere una tabella di routing con tutti i 4mld di indirizzi IP degli host mondiali, mentre così mi bastano gli indirizzi delle reti. Non può essere assegnato anche l'indirizzo con HostID con tutti 1 (quello con 255.255 insomma), perché è l'indirizzo di broadcast della rete: invio pacchetti a tutti gli host della rete se uso questo.

Come distinguo i bit di NetID e quelli di HostID? Agli indirizzi IP viene assegnata una netmask, una maschera che mi permette di identificare i bit che fanno parte del NetID: contiene tutti 1 nella parte relativa alla rete, mentre ha tutti 0 nella parte relativa all'host. Viene effettuata un'operazione di AND logico (se ho 1-1 rimane 1, altrimento 0) tra l'indirizzo in questione e la netmask, ottenendo l'indirizzo della rete.

Esempio indirizzamento IP
Ho indirizzo IP 130.86.15.64. Non viene mai mandato da solo ma con la sua netmask: viene specificato infatti come 130.86.15.64\16, ovvero i primi 16 bit appartengono alla netmask. Trasformo i numeri che compongono l'indirizzo IP in binario, ottenendo 130=10000010, 86=01010110, 15=00001111, 64=01000000.

In questo caso vale la netmask 11111111.11111111.00000000.00000000 (255.255.0.0): applicandola si ottiene il prefisso (decimale) 130.86, che rappresenta tutti i 2^{16} indirizzi che iniziano con quei bit.

Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

- **Indirizzamento classful**
  Inizialmente tutti gli indirizzi erano suddivisi in 5 classi:
  - Classe A: primo bit 0, sono tutti quelli con NetID di un byte
  - Classe B: primi due bit 10, 2 byte per NetID
  - Classe C: primi tre bit 110, 3 byte per NetID
  - Classe D: primi quattro bit 1110, riservati per il multicast
  - Classe E: primi 4 bit 1111, riservati per usi futuri

  Metà degli indirizzi globali sono di classe A, tutti quelli che iniziano per 1 si distribuiscono tra le altre 4 classi (2^{31} indirizzi). Gli indirizzi di classe B sono il 25% degli indirizzi totali, quelli di classe C 12,5% e classe D e E 6,25% a testa. Possono esistere 2^7=128 reti di classe A (guardiamo solo il NetID che ha un valore fissato) e in ogni rete posso avere 2^{32-8}=16.777.216 interfacce. Segue ogni volta per le altre classi, basta considerare la lunghezza di NetID e HostID corretta.

  Nella realtà le reti di classe A sono 126: quella con NetID 0 viene utilizzata da una macchina che invia un pacchetto senza conoscere il proprio indirizzo IP. Anche 127 è bloccato. Guardando il valore del primo numero dell'indirizzo IP posso capire di che classe fa parte: da 1 a 126 classe A, da 128 a 191 classe B, da 192 a 223 classe C, da 224 a 239 classe D e da 240 a 255 classe E.

  Esistono degli indirizzi speciali, assegnati a funzioni particolari:

Special Address	*Netid*	*Hostid*	*Source or Destination*
Network address	Specific	All 0s	None
Direct broadcast address	Specific	All 1s	Destination
Limited broadcast address	All 1s	All 1s	Destination
This host on this network	All 0s	All 0s	Source
Specific host on this network	All 0s	Specific	Destination
Loopback address	127	Any	Destination

  L'indirizzo con tutti 1 manda il pacchetto a tutti gli host della rete in cui mi trovo, ma viene bloccata la comunicazione dai router se tenta di uscire dalla rete. Il loopback address non fa neanche uscire dal mio computer il pacchetto inviato: esce dalla porta TCP di uscita e rientra da quella di entrata.

  Esistono anche indirizzi detti **pubblici o privati:** sono alcuni indirizzi utilizzabili da chiunque, ma che non possono essere associati a pacchetti che circolano nella rete pubblica. Sono suddivisi in 3 blocchi:
  - 10.0.0.0-10.255.255.255: 1 blocco di indirizzi di classe A (2^{24} indirizzi)
  - 172.16.0.0-172.31.255.255: 16 blocchi di indirizzi adiacenti di classe B (2^{20})
  - 192.168.0.0-192.168.255.255: 256 blocchi di indirizzi adiacenti di classe C (2^{16})

- *Meccanismo di inoltro dei pacchetti*
  Approssimiamo, per semplificare, che sotto il livello 3 che consideriamo (IP) ci sia solo un tipo di livello 2 (ad esempio Ethernet): ciò significa che ogni host è caratterizzato da un indirizzo IP e da un MAC Address. Nella maggior parte dei casi l'assunzione di prima è vera, una IP net corrisponde a un solo livello 2. Esistono diversi meccanismi di inoltro a seconda del fatto che io vada da IP locale a IP locale (inoltro diretto) o da IP locale a IP remoto (inoltro indiretto).

Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

Host	Router
Inoltro diretto, da IP locale a IP locale (stessa rete)	Inoltro diretto, riceve il pacchetto e si accorge che è destinato a una delle reti a cui è collegato
Inoltro indiretto, da IP locale a IP remoto (da una rete ad un'altra)	Inoltro indiretto, riceve il pacchetto e lo inoltra ma la rete a cui lo manda non è la rete di destinazione

L'inoltro negli host può essere diretto o indiretto, a seconda della presenza o meno di sorgente e destinazione nella rete locale. Vengono eseguiti nel modo seguente:

- **Inoltro diretto negli host (1)**: un host deve spedire un pacchetto a un altro host all'interno della rete. Applica la netmask e, se restituisce lo stesso valore della rete di cui fa parte anche lui (confrontando il valore restituito dalla netmask con il proprio indirizzo IP), non fa altro che inviarlo all'indirizzo di destinazione.

  In ciascun computer c'è una tabella di corrispondenza, per la rete locale, tra indirizzi IP e indirizzi MAC e viene costruita con un protocollo detto ARP. L'host che deve inviare il pacchetto consulta la tabella, trova l'indirizzo MAC corrispondente all'indirizzo IP a cui deve inviare il pacchetto, e lo invia a quell'indirizzo.

  Dentro il pacchetto IP troviamo Source IP e Destination IP e tutto il pacchetto IP viene "incapsulato" nella trama di livello 2, che contiene a sua volta il Source MAC e il Destination MAC. Nella trama quindi sono contenuti sia gli indirizzi MAC che gli indirizzi IP di sorgente e destinazione.

- **Inoltro indiretto negli host (2)**: un host vuole inviare un pacchetto a un altro host, non facente parte della sua rete. Non posso usare la tabella di corrispondenza tra MAC e IP perché non sono nella stessa rete. Devo quindi rivolgermi al router, al default gateway (unica via di uscita dalla rete): me ne rendo conto applicando la netmask e confrontando il risultato con l'indirizzo IP dell'host di partenza, vedendo che non corrispondono le reti di appartenenza.

  Conosco l'indirizzo IP del default gateway, quindi nella tabella di corrispondenza avrò il suo indirizzo MAC: invio il pacchetto al router. Il router cambierà gli indirizzi MAC presenti nella trama, senza modificare gli indirizzi IP che rimarranno quello dell'host di partenza e di quello di destinazione: quando gli arriva la trama e la inoltra ad un'altra rete mette come Source MAC il suo e come Destination MAC quello dell'host di destinazione (che conosce perché si interfaccia con la sua rete).

  L'inoltro nei router è detto destination based: l'inoltro IP è basato solamente sull'indirizzo di destinazione. Viene detto next-hop routing, perché nelle tabelle di routing è definito solamente il "salto" successivo verso la rete successiva per arrivare a destinazione. Nelle tabelle è presente l'indirizzo di destinazione, la netmask, il next hop, ecc. Viene eseguito come segue:

- **Inoltro diretto nei router (3)**: avviene quando un router è effettivamente collegato alla rete di destinazione.

- **Inoltro indiretto nei router (4)**: avviene quando un router deve mandare la trama lungo una certa strada per arrivare a destinazione, ma non è collegato direttamente alla rete di destinazione. Ho tutte le destinazioni nella tabella di routing: devo confrontare l'indirizzo IP con tutti quelli delle reti della tabella. Se il pacchetto in arrivo ha un indirizzo che non corrisponde a nessuna delle entrate della tabella lo inviamo al default address, ovvero quello con la netmask di tutti 0 e faccio inoltro diretto.

Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

Quando ottengo più corrispondenze, dopo l'applicazione della netmask, invio seguendo il longest prefix matching, ovvero l'indirizzo che ha più bit in comune con l'indirizzo del pacchetto in uscita (vedi esempi 2 e 3 sulle slide).

Quando arrivo in un router intermedio, non in quello che inoltra verso la destinazione, non ho una tabella di routing con 4 miliardi di input: viene fatta route aggregation, perché mi interessa sapere solamente l'indirizzo di rete e non quello di un singolo host. L'indirizzo della rete non è associato a nessun host ma mi serve per costruire le tabelle di routing.

- *Subnetting*

Mi serve per "ottimizzare" e non sprecare indirizzi: se devo assegnare 1000 indirizzi non uso una rete di classe B, spreco una marea di indirizzi. Non uso neanche 4 reti di classe C perché è inefficiente: ogni azienda avrebbe più indirizzi, ci sarebbero problemi di scalabilità. Possiamo suddividere una rete in più sottoreti (solitamente viene fatto con una rete di classe B, ma accade anche su reti di classe A oppure C). L 'indirizzo che contiene in NetID della rete (quello che ci dice in che classe siamo) più il NetID viene detto Extended Network Prefix.

Viene usato quando non bastano indirizzi di una certa classe oppure se una classe è sovradimensionata per l'uso che devo farne. L'idea di base è quella di non comporre l'indirizzo IP solamente con Host ID e Net ID, ma viene aggiunto un Subnet ID. Definisco il Subnet ID di lunghezza n bit: è come se definissi l'esistenza di 2^n reti di una nuova classe, con un Net ID di x+n (con x lunghezza Net ID a seconda della classe).

Aggiungendo, ad esempio, un Subnet ID da 5 bit a una rete di classe B (Net ID da 16 bit, compresi i due iniziali identificativi della classe), otterremo una rete /21 invece che /16: in ogni subnet possiamo avere 2^n indirizzi con n dimensione del Subnet ID. La corrispondenza indirizzo-subnet è fondamentale: non posso buttare lì un indirizzo normalissimo (ad es. 159.10.8.0), ma devo per forza specificare la netmask per capire in che sottorete sono (ad esempio 159.10.8.0/21, in questo caso sto specificando una sottorete). Se avessi indicato 159.10.8.0/16 avrei indicato un singolo host.

Esempio subnetting

Ho un host con indirizzo 159.100.9.18.
Scrivendo l'indirizzo in binario ottengo 10011111.01100100.00001001.00010010 e noto che inizia per 10: è in classe B. L'indirizzo di rete, data la netmask 255.255.0.0, corrisponde a 10011111.01100100.00000000.00000000\16 (159.100.0.0, ho applicato la netmask). In questa rete posso avere 2^{16}-2=65.534 host: sono anche troppi, decido di dividere in 32 sottoreti.

Devo riservare, per poter definire le 32 sottoreti, 5 bit, poiché 2^5=32. Faccio variare il campo SubnetID.
10011111.01100100.**00000**000.00000000 (quelli segnati sono i bit del SubnetID)
Alcuni esempi di subnet relativi all'esempio possono essere:
10011111.01100100.00000000.00000000 (159.100.0.0\21, uguale a prima ma è cambiata la Netmask)

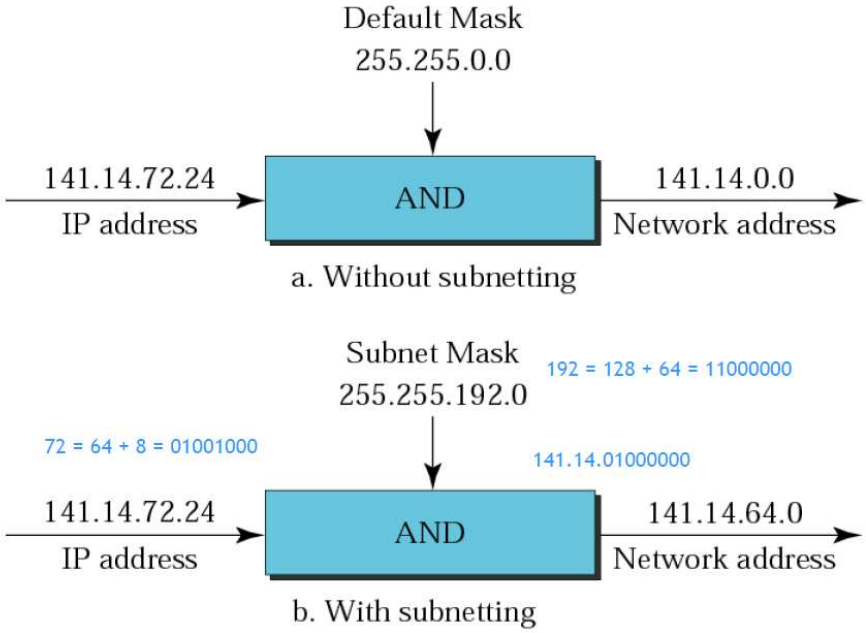
Indirizzo subnet	Binario	Indirizzo broadcast
159.100.0.0\21	10011111.01100100.00000000.00000000	10011111.01100100.00000111.11111111 (159.100.7.255)
159.100.8.0\21	10011111.01100100.00001000.00000000	10011111.01100100.00001111.11111111 (159.100.15.255)
159.100.16.0\21	10011111.01100100.00010000.00000000	10011111.01100100.00011111.11111111 (159.100.31.255)
159.100.248.0\21	10011111.01100100.11111000.00000000	10011111.01100100.11111111.11111111 (159.100.255.255)

Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

Il numero di host della subnet sarà sempre 2^n con n numero di bit riservati per il SubnetID. Se mi servisse una rete con 4 indirizzi (non 4 host) uso una subnet mask \30, che mi lascia gli ultimi due bit: posso metterci solo 2 host perché due indirizzi sono riservati, uno broadcast e l'altro come indirizzo della rete. La subnet \31 non esiste, non ci sono indirizzi utilizzabili.

Esempio subnetting in classe B

Il subnetting tipico in classe B (rete \16) è quello con una netmask \24 per ottenere 256 sottoreti. Viene usato solitamente perché non si "spezza" un byte.



Quando applico la subnet mask faccio un'operazione di && logico: fino a quando nella subnet ho 1 mantengo i bit dell'indirizzo come sono, quando nella subnet ho 0 pongo tutto a 0 nell'indirizzo della rete e ciò che ricavo dall'applicazione della subnet stessa è l'indirizzo effettivo della rete.

Esempio subnetting 2

Indirizzo della rete iniziale	Classe	Indirizzo in binario
128.234.0.0\16	B	10000000.11101010.00000000.00000000\16

Devo creare almeno 1000 sottoreti: devo riservare 10 bit in modo da poter definire 2^{10}=1.024 sottoreti. Riserverò 10 bit per il subnet ID.

Subnet	Binario
128.234.0.0\26	**10000000.11101010.00000000.00**000000
128.234.0.64\26	**10000000.11101010.00000000.01**000000
128.234.0.128\26	**10000000.11101010.00000000.10**000000
128.234.0.192.\26	**10000000.11101010.00000000.11**000000
128.234.1.0\26	**10000000.11101010.00000001.00**000000
128.234.255.192\26	**10000000.11101010.11111111.11**000000

Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

Es. subnetting 3

Rete	Classe	Binario
141.14.0.0\16	B	10001101.00001110.00000000.00000000

Voglio 4 sottoreti, riservo 2 bit come subnet ID

Rete	Binario
141.14.0.0\18	**10001101.00001110.00**000000.00000000
141.14.64.0\18	**10001101.00001110.01**000000.00000000
141.14.128.0\18	**10001101.00001110.10**000000.00000000
141.14.192.0\18	**10001101.00001110.11**000000.00000000

- *Note sul subnetting*

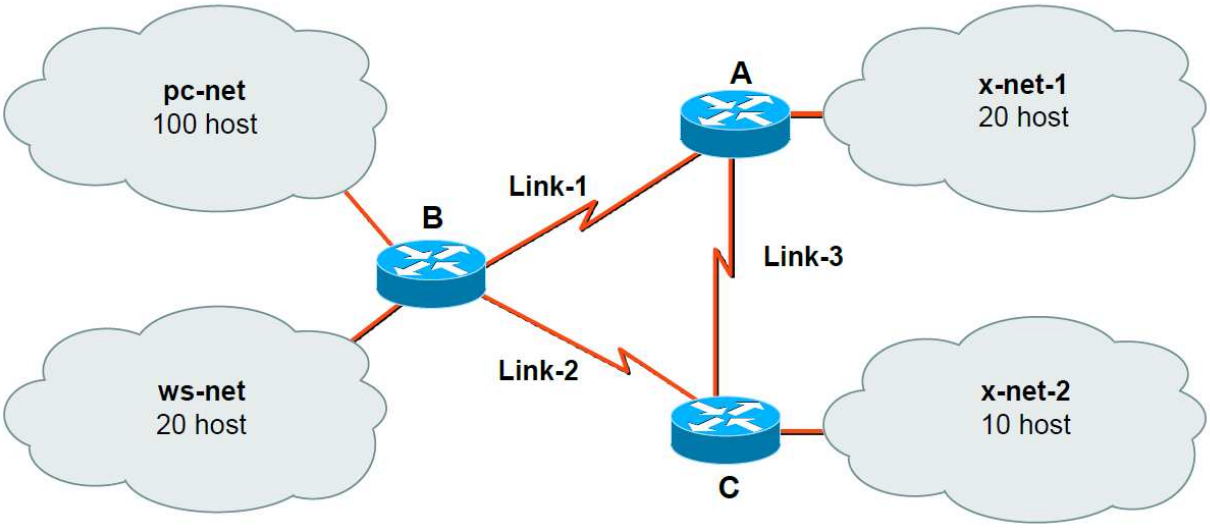
Fino a che sono nella rete internet vedo una rete di classe C: quando entro nella rete scopro l'esistenza delle subnet e dei router che mi ci reindirizzano. Se ad esempio mi interessa vedere la rete 162.12.0.0, quando ci arrivo scopro che è subnettata: fino a che non arrivo al router interno alla rete che mi interessa non serve sapere la subnet, ma mi basta fare l'indirizzamento sulla rete "grossa", mi basta sapere il NetID.

Se dovessi sapere sempre il subnet ID dovrei comunicare a tutto il mondo l'indirizzo, invece così vado per step e comunico solamente la rete di classe A/B/C in cui mi trovo. Fino a che il pacchetto arriva al "confine" della rete non si sa niente della subnet, una volta che arrivo alla rete ci sarà il router che indirizzerà correttamente il pacchetto alla subnet corretta.

- *VLSM*

VLSM= Variable Length Subnet Mask. Non suddivido la rete nelle sottoreti che mi servono, ma applico in cascata più subnet mask per ricavare più sottoreti.

Esempio VLSM



Ogni link che connette tra loro due router è di per sé una sottorete: mi serve identificarli perché ogni interfaccia deve avere un nome per i router. Link 1, link 2 e link 3 hanno 2 host a testa: anche il link ha bisogno di una sottorete per poter associare un indirizzo a ogni router. La rete pc-net avrà una subnet \25 (128-2 perché tolgo i riservati), perché mi permette di definire 126 indirizzi. Ws-net sarà \27, così come x-net-1. x-net-2 avrà subnet mask \28 perché sufficiente a definire tutti gli indirizzi. Per i link è sufficiente \30.

Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

Usando VLSM facciamo:

Rete/sottorete	Indirizzo	Binario
Rete in classe C	192.168.1.0\24 (254 indirizzi)	**11000000.10101000.00000001**.00000000
Pc-net	192.168.1.0\25 (126 indirizzi)	**11000000.10101000.00000001.0**0000000
Altra rete (suddivido ulteriormente)	192.168.1.128\25	**11000000.10101000.00000001.1**0000000
Ws-net	192.168.1.128\27 (30 indirizzi)	**11000000.10101000.00000001.100**00000
x-net-1	192.168.1.160\27 (30 indirizzi)	**11000000.10101000.00000001.101**00000
Altra rete (suddivido ulteriormente)	192.168.1.192\27	**11000000.10101000.00000001.110**00000
Libera per usi futuri, non ci serve	192.168.1.224\27	**11000000.10101000.00000001.111**00000
x-net-2	192.168.1.192\28 (14 indirizzi)	**11000000.10101000.00000001.1100**0000
Altra rete (suddivido ulteriormente)	192.168.1.208\28	**11000000.10101000.00000001.1101**0000
Link1	192.168.1.208\30 (2 indirizzi)	**11000000.10101000.00000001.110100**00
Link2	192.168.1.212\30 (2 indirizzi)	**11000000.10101000.00000001.110101**00
Link3	192.168.1.216\30 (2 indirizzi)	**11000000.10101000.00000001.110110**00
Libera per usi futuri	192.168.1.220\30 (2 indirizzi)	**11000000.10101000.00000001.110111**00

Per le due x net mi bastano reti piccole, quindi posso riservare al netID anche 2 bit in più, arrivando a \27. L'indirizzamento funziona come sempre, con i router.

- *Supernetting*

A volte non posso prendere le subnet che mi servono nella classe B, ma devo prenderne più di una da una rete di classe C: la rete aziendale verrà rappresentata da più subnet e non è ottimale. Posso metterle insieme mediante il supernetting: ho tante reti piccole, ne faccio una grande. Aggiungo all'hostID dei bit presi dal netID.

- **4 reti di classe C (netmask /24) con indirizzi contigui**
  - 213.2.96.0 — 11010101.00000010.01100000.00000000
  - 213.2.97.0 — 11010101.00000010.01100001.00000000
  - 213.2.98.0 — 11010101.00000010.01100010.00000000
  - 213.2.99.0 — 11010101.00000010.01100011.00000000
- **Supernet mask**
  - 255.255.252.0
- **Supernet address: 213.2.96.0/22**
  - 11010101 . 00000010 . 011000 00 . 00000000

Definisco una nuova netmask, in questo caso supernet mask, che mi consente di mettere insieme reti piccole in una sola rete più grande identificata da una sola supernet mask. Troviamo, invece di 4 reti \24 una sola rete \22.

Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

- **Indirizzamento classless**

Si basa su CIDR, Classless Inter-Domain Routing. Rappresenta una soluzione al problema causato dalle tabelle di routing e tampona il problema dell'esaurimento degli indirizzi. La soluzione definitiva a questo problema sarebbe l'adozione di IPv6, con indirizzi a 128 bit, ma non è semplice.

Con CIDR non consideriamo il concetto di classe, ma suddividiamo i 2^{32} indirizzi senza nessun vincolo, facendo subnetting e supernetting se e quando necessario. Per definire un blocco di indirizzi useremo un primo indirizzo x.y.z.w/n, con una maschera di rete di n bit che andrà a definire 2^{32-n} indirizzi adiacenti a partire dall'indirizzo specificato.
Ad esempio da 192.168.0.0 a 192.168.255.255 ci sono 2^8*2^8=65.536 (2^16) indirizzi possibili, ovvero troviamo $2^{32-n} = 2^{16}$e quindi n=16. Posso creare indirizzi di rete /n classless simili tra loro ma, ovviamente, indipendenti dalla classe alla quale apparterrebbero.

Il protocollo di routing usato deve trasportare il prefisso di rete insieme alle informazioni che deve veicolare: tutti i router adottano longest prefix matching per l'instradamento, ma per consentire route aggregation è necessario che gli indirizzi siano assegnati in maniera significativa (subnet vicine e indirizzi adiacenti).

- *NAT*

In linea di principio un amministratore ha una rete con n utenti (n indirizzi) ma non tutti escono dalla rete: su una rete piccola non ha senso, ma su una enorme (es. operatore) succede molto spesso. Vengono quindi utilizzati degli indirizzi privati che vengono visti solamente all'interno della rete, non escono, che vengono assegnati solamente per rimanere all'interno della rete stessa. Quando ho bisogno di uscire dalla rete il NAT (Network Address Translation, è una tabella) deve darmi un indirizzo per andare fuori dalla rete. Se la maggior parte degli utenti vuole uscire dalla rete non ho vantaggi reali ad applicare il NAT, perché avrei lo stesso numero di indirizzi tra pubblici e privati (ma avrei facile scalabilità e adattabilità, se cambio indirizzo pubblico non devo riconfigurare tutta la rete privata).

- *NAPT*

Network Address Port Translation: evoluzione del NAT, sfrutta il numero di porta per individuare la connessione associando ad un host interno la stessa coppia di indirizzi pubblici (IP e port number). Posso indirizzare all'interno della rete 2^{16}=65.536 indirizzi con l'indirizzo della mia rete mediante il NAPT: veniva fatto specialmente dagli operatori che non avevano ricevuto l'indirizzo dalla IANA.

- *IPv6*

I 32 bit di IPv4 non bastano: viene creato IPv6. Non è diffuso equamente nel mondo, ci sarà il 10% nel mondo e basta, non ha ancora preso molto piede. Lo spazio degli indirizzi viene aumentato a 128bit, quindi potremmo avere 2^{128}=3,4*10^{38} indirizzi (sono tantissimi). Vengono tolti alcuni campi dell'header poco usati come il checksum, la gestione delle opzioni avviene in modo diverso, viene introdotto il concetto di flusso e viene fatta autoconfigurazione e sicurezza. Non si passa da classful, direttamente classless.

- *Protocollo TCP/IP*

Ci sono dei protocolli accessori che servono per far funzionare IP.

  - ICMP: ha compiti di gestione degli errori, diagnostica, ecc.
  - Rimane comunque legato al livello 3 ma viene posto sopra IP, anche se "viaggia" all'interno dei pacchetti IP: è considerabile un "cliente" del livello IP. Il trasporto del messaggio ICMP avviene all'interno del pacchetto IP. È formato da un campo type (8 bit, serve per specificare che tipo di operazione devo fare), un campo code (8 bit) e un checksum (16 bit).

Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

Alcuni messaggi ICMP sono noti:

- Echo: è il messaggio che viene usato per fare ping, ovvero verificare la raggiungibilità di un host o un router. Viene utilizzato mediante Echo-request e Echo-reply per chiedere e ricevere (viene restituito 1 nel caso lo script vada a buon fine). Se faccio una "cascata" di Echo-request (logicamente parlando) ottengo traceroute: mi viene data una risposta da tutti i router intermedi fino al router finale.

- OSPF: serve per riempire le tabelle di routing. A monte di OSPF esiste un algoritmo di cammino minimo che serve per scegliere la strada più breve
- ARP e RARP: servono per riempire la tabella di corrispondenza IP/MAC con gli indirizzi di tutti gli host della rete locale.
  - ARP: viene usato per l'inoltro nella rete locale, utile per le corrispondenze IP/MAC. In pratica riempie una tabella di routing grazie ad un pacchetto ARP.

    La stazione richiedente manda un broadcast ARP, se la stazione cercata è presente nella rete comunica il suo indirizzo MAC e posso quindi inviare il messaggio. Non lo richiedo ogni volta ma esiste una ARP cache che mantiene in memoria gli indirizzi per un certo lasso di tempo.

    In ARP è presente un campo hw type, che comunica che tipo di hardware utilizzo, un campo protocol type (sempre IP), hardware length (lunghezza in byte dell'indirizzo di livello 2, 6 byte), protocol length (lunghezza in byte dell'indirizzo di livello 3, 4 o 32 byte), operation (che tipo di operazione è, richiesta o risposta), sender (indirizzo IP e MAC di chi invia) e target (IP e MAC dell'host che sto cercando nella rete).

- *Routing IP*

Come faccio a scegliere il percorso migliore per arrivare ad un host particolare? Esistono degli algoritmi che permettono di scegliere il cammino ideale. L'algoritmo deve essere robusto (evitare i fallimenti della rete trovando un'alternativa), stabile (adattarsi alla presenza di una nuova rete) e ottimale (sempre scegliere la via più veloce). Gli algoritmi di routing vengono applicati sui pacchetti IP.

Possiamo classificare gli algoritmi sulla base del nodo che opera la decisione.

- **Centralizzato**: c'è un unico centro di controllo per tutte le decisioni della rete. Nelle reti di comunicazioni sta prendendo piede questa tipologia (SDN)
- **Distribuito**: ogni router prende la sua decisione sulla base delle informazioni disponibili, cooperando per determinare il migliore instradamento. Ciascun elemento di instradamento (router) prende la decisione localmente con le informazioni che ha in quel momento, ognuno localmente prende la decisione che al momento risulta migliore
- **Isolato**: la decisione viene presa da un solo router, non decide solamente il next hop ma nel pacchetto mette anche tutto il cammino che il pacchetto stesso deve percorrere per arrivare a destinazione. Il cammino viene stabilito all'inizio e non viene "creato" mano a mano dai router con le informazioni disponibili. I router successivi al primo si limitano a fare un inoltro verso la destinazione nel modo specificato dal primo.

Posso classificare gli algoritmi anche in base alla presenza o meno di una tabella, come quella di routing:

- **Protocolli senza tabella**
  - Random: mi arriva un pacchetto, sicuramente non lo mando all'interfaccia da dove mi è arrivato ma con equiprobabilità lo invio in modo casuale a un'interfaccia. È comodo perché economico, minimamente complesso e robusto, viene utilizzato specialmente se nei nodi ho pochissima capacità di calcolo. Non è ottimale, non si percorre praticamente mai il percorso più breve, e non si conosce il ritardo con cui arriverà a destinazione il pacchetto.

Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

Si può aumentare l'intelligenza dell'algoritmo proporzionando la probabilità con la quale un'uscita viene scelta in base alla capacità disponibile sul link corrispondente (vedi slide per formula). Nella rete IP non ha senso.

- Selective random: metto una regola fissa che mi dice da che parte devo muovermi, selezionando una direzione verso cui andare in base all'interfaccia di entrata. È "meno casuale" ma rimane comunque random e non è più completamente isolato (richiede la disponibilità di una tabella delle adiacenze, se il criterio di instradamento è l'adiacenza all'entrata).

- Flooding: il pacchetto viene mandato replicato su tutte le uscite tranne quella da cui è arrivato. Aumenta tanto il traffico interno ma massimizza la probabilità di far arrivare il pacchetto a destinazione, dando la massima priorità alla trasmissione.

  È possibile limitare il flooding, mantenendo una memoria nei nodi dei pacchetti che ha già ricevuto e inoltrato in modo da non inoltrarlo nuovamente: servono però dei nodi con memoria e capacità di calcolo. Posso anche fare selezione, decidendo un sottoinsieme di tutte le uscite (selective flooding).

- Source routing: dalla sorgente decido l'intero percorso, è un'implementazione dell'algoritmo isolato. Il nodo sorgente decide il cammino, specificando nel pacchetto la sequenza di router da seguire per arrivare a destinazione. Per determinare la strada da seguire si può utilizzare un percorso fornito da un path server (centralizzato, semplice ma inaffidabile) oppure con path discovery (mando pacchetti in flooding che attraversano tutti i nodi necessari per arrivare a destinazione, viene scelto il pacchetto che ci mette meno o che attraversa meno nodi e determino in modo isolato il percorso da seguire).

- **Protocolli con tabella**: è il tipo seguito da IP, viene solitamente scelto il cammino a costo minimo (non sempre è il cammino più veloce, bisogna definire una metrica di costo da minimizzare che può essere il ritardo dei link, il numero di link attraversati, il numero medio di pacchetti in coda, oppure voglio massimizzare l'affidabilità del percorso, ecc.). Nelle tabelle di instradamento viene specificato il next hop.

  Questi protocolli si suddividono in protocolli di instradamento statico o dinamico:
  - Dinamico: la tabella viene modificata in base allo stato della rete e alle esigenze. I protocolli di routing in IP sono dinamici, possono cambiare le entry delle tabelle di routing nel tempo in base a guasti, cambiamenti nella topologia, carico di rete e congestione.
    - Distance vector: non lo facciamo
    - Link state: detto anche Dijkstra, è l'algoritmo dinamico utilizzato per fare instradamento in IP. Sfrutta, per costruire la tabella in ogni nodo, LSA (link state advertisement), un pacchetto che serve per conoscere tutti i router della rete e quelli adiacenti. L'algoritmo ha un tempo di convergenza dato, così come il numero di passaggi per far passare in tutti i nodi il LSA.

      Il ritardo massimo è legato al diametro della rete. La distanza misurata viene comunicata a tutti i nodi in modo flooding: ogni nodo ha visibilità completa della rete, così può costruire i percorsi a costo minimo verso tutti gli altri nodi.

      Nelle slide il nodo C riceve il LSA dai nodi precedenti, conoscendo tutti i link della rete e quelli adiacenti agli stessi (comunicati mediante LSA). Mano a mano che arrivano LSA costruisco la topologia e la aggiorno con il passare del tempo, perché tipicamente vengono mandati ogni tot di tempo.

Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

Se in ciascun router conosco la topologia della rete, posso applicare l'algoritmo dei cammini minimi (Dijkstra) che mi restituisce l'albero dei cammini minimi (minimum spanning tree) con il quale posso costruire la tabella di routing con i cammini ideali. In ogni router viene generata in questo modo. Le tabelle di routing vengono riempite così.

- Statico: tipicamente viene fatto manualmente, l'amministratore di rete compila la tabella decidendo il percorso mediante istruzioni opportune. Non può funzionare su una rete grande come Internet, ma può funzionare su una rete piccola come quella di un'azienda.

- *Algoritmo di Dijkstra*

Genera un albero perché il grafo dei cammini non è mai chiuso, non presenta mai un ciclo. Genera un albero relativo al cammino minimo (MST, Minimum Spanning Tree).
Viene applicato al generico nodo s della rete e presenta una serie di dati da utilizzare:

- s=generico nodo sorgente della rete
- N=nodi della rete
- M=insieme dei nodi che attualmente fanno parte del minimum spanning tree
- V(M)= insieme dei nodi vicini ai nodi dello spanning tree, nodi adiacenti
- d_{ij}=costo della via diretta da i a j, pari a infinito se manca il link diretto
- D_j=via a costo minimo per andare da s a j, viene aggiornato mano a mano
- P_j=nodo predecessore del nodo j

Inizialmente avrò M={s}, perché è l'unico che conosco fin dall'inizio e che non può non fare parte del MST. All'inizio dal nodo s vedo solamente i nodi adiacenti, così come pongo a infinito la distanza tra s e i nodi non adiacenti che ancora non conosco. La prima operazione che faccio è aggiungere un nodo a M (e quindi aggiungendolo a MST): seleziono tra tutti i nodi adiacenti quello con il link a costo minore (solitamente distanza minore, ma la metrica di costo potrebbe anche essere diversa).

Ad ogni step aggiungo un nodo all'insieme M, calcolando ad ogni passo le distanze per l'intero insieme M (non solo con i nodi adiacenti al nodo s, ma anche con quelli che posso raggiungere muovendomi in M). Ogni volta che aggiungiamo un nodo a M controlliamo la distanza di j da s, perché potremmo scoprire nuove vie con cui raggiungere j a costo minore: prendiamo il minimo tra il costo già presente e quello appena calcolato. Procediamo per iterazione fino a coprire tutti i nodi della rete, generando l'albero a costo minimo.

È impossibile applicare Dijkstra a tutto Internet, perché risulta suddivisa in Autonomous Systems (AS, insieme di sottoreti connesse, possibilmente gestite da un'unica entità).

- *Operatori e protocolli di routing*

Ogni operatore lancia nella rete la sua versione di algoritmo per trovare il minimum spanning tree. È un sistema complesso, si possono trovare sulla stessa rete componenti che sfruttano algoritmi diversi.

- *OSPF*

Open Shortest Path First. È il protocollo che implementa praticamente Dijkstra sulle macchine che si occupano di routing IP. L'update può avvenire in modo periodico (ogni 30 secondi solitamente) oppure ogni volta che accade un tale evento, come la modifica dei costi dei link presenti nella rete. È possibile aggiornare MST solo in occasione di modifiche come quest'ultima, evitando di inviare update periodici (che hanno comunque un costo). I messaggi OSPF sono contenuti nel pacchetto IP e, per aggiornare le tabelle di routing, il messaggio viene inviato con selective flooding.

Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

Livello 4: strato di trasporto
Svolge lo stesso ruolo che il livello 2 ha con il livello 1, ovvero corregge gli errori ma del livello 3. IP non si occupa di correggere fuori sequenza, perdita o ripetizione di pacchetti. Il livello 4 non mette in comunicazione solo due macchine, ma un processo di una macchina con un processo in un'altra macchina. Un processo è un programma in esecuzione su un host (proprio i programmi che girano sul computer e che comunicano all'interno di una macchina attraverso il SO): spesso i processi devono comunicare con macchine diverse, è necessario gestire la comunicazione tra processi analoghi in macchine diverse.

L'insieme di procedure che consentono all'applicazione di comunicare con la rete viene detto socket e lo scambio di messaggi tra i protocolli applicativi avviene sfruttando i servizi dei livelli inferiori, mediante SAP (Service Access Point, mette in comunicazione il livello applicativo e quello di trasporto nell'architettura TCP/IP).

Non è sufficiente IP per identificare un processo di una macchina, perché mi consente solo di identificare il singolo host mentre il processo potrebbe essere in esecuzione su più host. Oltre all'IP dell'host devo utilizzare un numero di porta, associato al processo in esecuzione. Al livello 4 l'identificatore dei processi sarà una combinazione dei due, essendo di fatto univoco: questo livello è presente solamente negli apparati terminali, guardandolo vedo solo la connessione end-to-end ignorando tutto ciò che c'è nel mezzo.

Su un host possono essere in esecuzione più applicazioni: spetta quindi al livello 4 svolgere la funzione di muliplexing/demultiplexing, gestendole mediante indirizzi contenuti nelle proprie UI. Questi indirizzi, lunghi 16 bit, sono detti porte: possono assumere valori da 0 a 65535, alcune note sono 21 (FTP) o 80 (WWW). Le porte fino alla 49151 sono dette Registered e necessitano di autorizzazione per essere usate, mentre quelle successive sono libere e solitamente vengono usate per brevi lassi di tempo.

Ogni processo, quando viene associato ad una porta, viene associato dal SO ad una coda di entrata e ad una coda di uscita: è presente una funzione di buffering dei dati.

- **Servizi di trasporto**

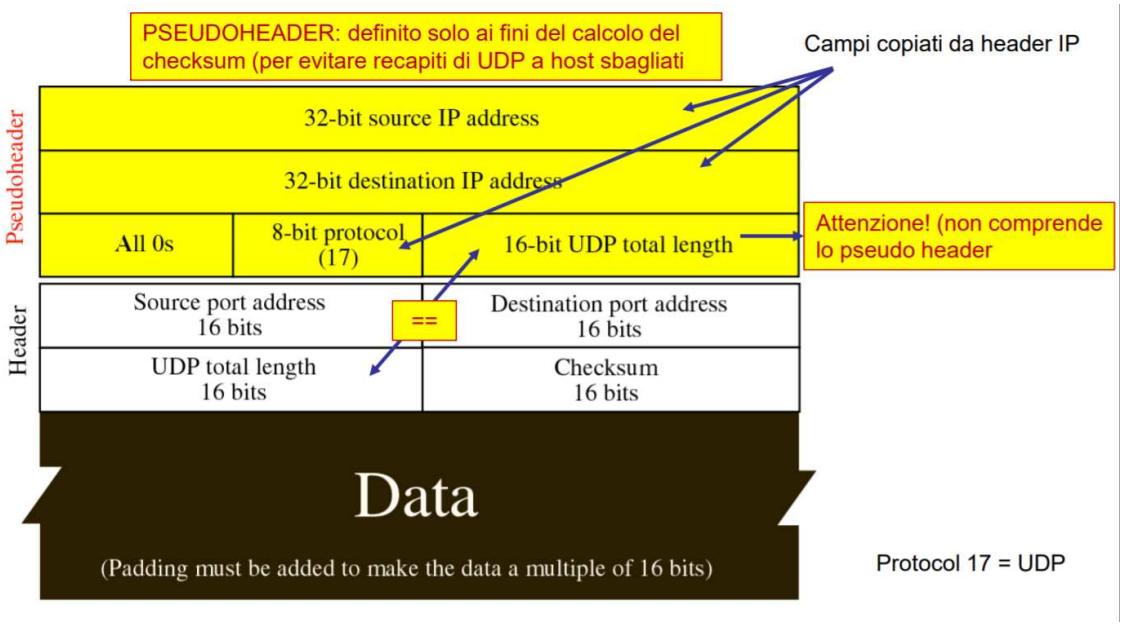
In generale il servizio di rete non è affidabile (best effort, fa del proprio meglio): un servizio di trasporto quindi può essere affidabile (garantisce la consegna dei messaggi nell'ordine corretto), oppure non affidabile (esegue multiplexing ma non da garanzie), connection-oriented o connectionless. In Internet sono definiti due tipi di trasporto, UDP e TCP.

UDP non chiede permessi, invia i pacchetti e basta e non corregge nulla in caso di errori. È utile in situazioni in cui è utile non aspettare un riscontro, come lo streaming video, perché consente di non far bloccare lo streaming stesso. UDP fa multiplazione, diversi processi della macchina sono associati a diverse porte e quindi deve gestirli per mezzo di socket. Oltre a ciò si occupa di fare un checksum, ovvero verifica che i bit inviati siano corretti (solo rivelazione, non correzione). UDP ha alcuni vantaggi:

- Minore latenza, non faccio controlli e devo solo trasferire
- Maggiore semplicità, poche regole e complessità bassa
- Minore overhead

Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

Un messaggio applicativo viene incapsulato esattamente in un pacchetto UDP e trasportato da un singolo pacchetto IP. L'header di UDP è lungo 8 byte ed è così composto



Il campo checksum non protegge solo il segmento UDP. Nel campo di checksum viene messo il complemento a 1 della somma in complemento a 1 del segmento UDP e dello pesudoheader allineati a 16 bit. Per vedere cosa c'è dentro vedi slides.

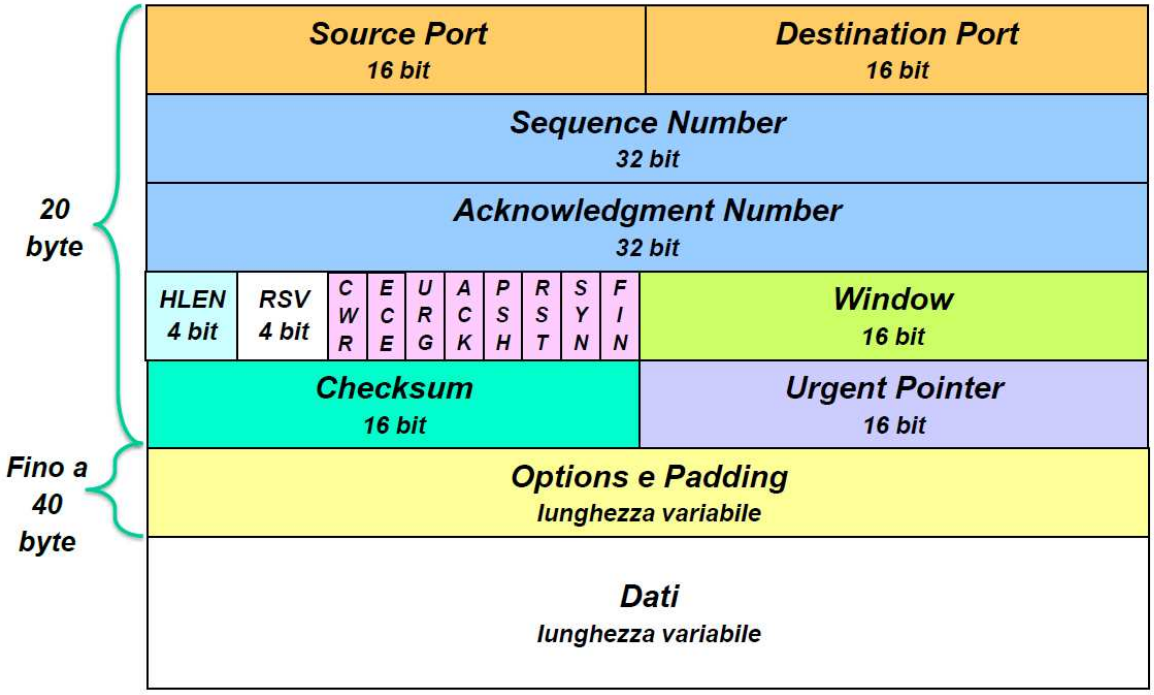
Con TCP, se devo trasferire un file, chiedo permesso per poter trasferire, con certe parametrizzazioni, e attendo risposta. A risposta ottenuta comunico che invierò. TCP si prende in carico la "correzione" delle informazioni, nel caso arrivino fuori sequenza o se alcuni pacchetti finiscano persi. È sicuro e affidabile.

- *Calcolo Checksum UDP*
  Per calcolare il checksum UDP procediamo come segue:
  - Prendiamo il segmento UDP rappresentato da una stringa di bit e lo suddividiamo in blocchi da 16 bit
  - Sommiamo i primi due blocchi (somma numeri binari): otterremo probabilmente un bit di riporto. Aggiungiamo il bit di riporto al bit meno significativo, sommandolo (bit meno significativo ovvero bit più a destra) e ottenendo una somma "modificata" rispetto a prima
  - Sommiamo il terzo blocco e ripetiamo. Così via per tutti gli altri eventuali blocchi, aggiungendo un padding se necessario (tanti 0 quanti servono)

Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

- Finita la somma dei blocchi eseguiamo il complemento a 1 della sequenza di 16 bit ottenuta e otteniamo il checksum, che viene inviato insieme al resto dei dati
- In ricezione viene effettuata nuovamente la somma in blocchi da 16 bit come prima: sommando la stringa di bit ottenuta al checksum si verifica la correttezza del messaggio (se sono tutti 0). Se si ottengono degli 1 c'è qualche errore

- **<u>TCP</u>**
  TCP è connection-oriented e affidabile. Il tempo di trasferimento varia in base alla rete e quindi TCP si deve adattare ai cambiamenti (congestione, blocchi, ecc.). Oltre a quello che fa UDP TCP fa anche consegna in sequenza, controllo di flusso e di perdita e controllo di congestione (congestione è con la rete, flusso è con il destinatario in modo end-to-end).



- Riga 1: contiene le porte della sorgente e della destinazione, 32 bit complessivi
- Riga 2: un numero che va da 0 a 2^{32}, rappresenta il numero di sequenza del primo byte del payload. Viene preso un numero casuale tra 0 e 2^{32}, associato al primo byte del file da trasferire, per poi trasferire lo stesso seguendo questa numerazione. Quando bisogna trasferire un altro file si ripete
- Acknowledgment number: numero di sequenza del prossimo byte che si intende ricevere, sensato solo se il flag ACK è valido. Riga di 32 bit
- HLEN: contiene la lunghezza complessiva dell'header TCP, rappresentata in numero di "parole" da 32 bit. Campo di 4 bit
- RSV: 4 bit riservati
- Window: specifica la dimensione della finestra di ricezione corrente, può essere aggiornata nel corso della trasmissione. Campo di 16 bit
- Checksum: corrisponde a quello di UDP
- Quelli in rosa sono dei campi di flag che assumono ciascuno valore 0 o 1 e hanno ognuno significato diverso.
- Option e padding hanno sempre la stessa funzione, vediamo una sola opzione.
  - Maximum Segment Size (MSS): definisce la dimensione massima del segmento di soli dati. È la lunghezza che viene proposta tra i parametri che TCP comunica quando "chiede" per la trasmissione. Di norma la lunghezza di default è uguale alla MTU del livello 2 tolti gli header, quindi una MSS di 536 byte.

Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

ISN (Initial Sequence Number) viene preso casualmente tra 0 e 2^32: indica il primo byte del pacchetto che devo inviare. Se ho un pacchetto con ISN pari a 60000 lungo 1000byte, il seguente potrà avere ISN uguale a 61000 nel nodo A. Nel nodo B, prima ricevente, pesco un altro ISN casuale per reinviare il pacchetto, non sarà uguale. I pacchetti successivi avranno ISN incrementale come nel nodo precedente (es. se il primo ha ISN=100000 ed è da 1000byte, il seguente avrà ISN=101000). Al livello 4 si lavora in byte, al livello 2 si lavorava per trame.

- *Gestione delle connessioni*

TCP è connection oriented: prima del trasferimento dati (solitamente connessione client-server) è necessario instaurare una connessione con una segnalazione. Per instaurare la connessione (setup) viene seguita la procedura del "three-way handshake", utilizzando i bit SYN e ACK. Il server effettua una passive open: comunica a TCP locale che può accettare nuove connessioni. Il client effettua una active open, ovvero comunica a TCP locale che è pronto a trasmettere verso un socket remoto.

Il client TCP prende un ISN casuale e invia un messaggio di sincronizzazione (flag SYN=1) al server, aggiungendo eventuali parametri della connessione nel messaggio. Nel caso questo messaggio vada perso viene effettuato subito un nuovo setup. Se invece il sync viene ricevuto correttamente, il server invia un segmento SYN/ACK (con i due flag =1) che contiene anche un AN (Ack number) con valore subito successivo all'ISN del messaggio di sync inviato dal client: così facendo viene riscontrato e viene comunicato qual è il prossimo byte atteso.

Il client riceve SYN/ACK e invia un ACK relativo ad esso, aggiungendo nel payload i primi dati della connessione con ISN del primo byte e comunicando anche la dimensione della finestra di ricezione da imporre al server. Al termine del trasferimento la connessione viene chiusa in modo analogo (teardown): il TCP che chiude la connessione invia un messaggio di fine (flag FIN=1) con gli ultimi dati e il TCP ricevente invia un riscontro (ACK) per confermare.

- *Controllo di flusso e trasporto dei dati*

Il TCP ricevente si occupa di controllare il flusso del TCP trasmittente: l'unico obiettivo è fare in modo che chi invia invii una quantità di dati che il ricevente può effettivamente ricevere. Da entrambe le parti è presente un buffer: il buffer di ricezione accumula i byte ricevuti ma non ancora assorbiti dall'applicazione, quello di trasmissione accumula i byte che attendono di essere trasmessi. La finestra di trasmissione di TCP è scorrevole, non periodica (non si ripete, parte dall'ISN e avanza fino alla fine, non è circolare).

Se tutto va bene invio, il byte viene ricevuto, viene inviato un riscontro dal ricevente e la finestra di invio scorre in avanti. Quando mando un ACK al trasmettitore includo un AN pari al primo spazio della finestra di ricezione (riscontro del byte y) e comunico anche la dimensione della finestra di ricezione rimanente (campo Window, la dimensione è indicata con w). I byte che rientrano nella finestra di ricezione possono essere memorizzati nel buffer, anche se arrivano fuori sequenza: al massimo posso ricevere il byte di numero $(y+w)-1$.

In invio posso inviare tutti i byte compresi nella finestra, quindi da y (ultimo byte non ancora riscontrato) a (y+w)-1 (ultimo byte inviabile in attesa di un ACK). Non è detto che io debba inviare tutti i byte: invio fino ad un certo SN (Sequence Number), pari a x-L+1 (x ultimo byte inviato, L numero di byte trasmessi con l'ultimo segmento TCP). I segmenti che contengono i byte da y a x sono memorizzati in un buffer di ritrasmissione: se vengono riscontrati vengono eliminati, altrimenti alla scadenza del timeout viene ritrasmesso il primo segmento presente nel buffer.

La finestra di ricezione (Receiver Window, RWND) si estende dall'ultimo byte ricevuto e riscontrato (y-1, escluso) fino alla fine del buffer: il buffer può riempirsi, ma la dimensione della RWND viene segnalata in ogni segmento inviato dal ricevitore al trasmettitore. La finestra di trasmissione (Sending Window, SWND) si estende invece dal primo byte non riscontrato (y, incluso) fino all'estremo destro della RWND. Il trasmettitore mantiene traccia dei dati non ancora riscontrati e della dimensione della RWND. Può accadere di avere SWND<RWND per criteri molto stringenti.

Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

Viene definita una flightsize, ovvero il numero di byte che ho inviato e che non è ancora stato riscontrato (sono appunto "in volo", potrebbe essere che sono già stati riscontrati ma il riscontro non è ancora arrivato), pari a x-y+1.

Esempio evoluzione RWND
Ho una receiving window da 300 byte vuota: invio w=300. Arriva un pacchetto di 100 byte, che occupa il suo spazio lasciando ancora 200 byte liberi. Parte un pacchetto che comunica w=200. Arriva un altro pacchetto da 100 byte, poi invia una comunicazione con w=100. L'applicazione assorbe i 200 byte presenti nella receiving window, comunico w=300 perché si è svuotata.

Stessa cosa nella finestra di invio: invio 100 byte su una finestra da 300 byte comunico che ho spazio per 200 byte, insieme con l'ACK del pacchetto successivo. Mano a mano che il ricevitore evolve manda informazioni alla sorgente sul come dimensionare la finestra di trasmissione, in modo da non creare congestione nella receiving window. Mano a mano che si occupa la finestra di ricezione la finestra di invio si riduce: quando si libera e viene comunicato la sending window torna alla dimensione precedente.

A volte si creano situazioni di "silly window", ovvero la finestra di invio si comporta in modo strano rallentando la connessione: esistono algoritmi che lo risolvono ma non li copriamo.

- *Ritrasmissione e errori*

Non possiamo usare GBN e SR come al livello 2 perché il round trip time (RTT) non è costante: i segmenti TCP viaggiano attraverso una rete e non attraverso un link, i ritardi che subiscono sono per la maggior parte variabili. Non posso pensare di mandare un NACK quando ricevo l'ACK del pacchetto successivo rispetto a quello che dovrei ricevere se la trasmissione fosse corretta: non posso desumere da ciò che il pacchetto precedente sia perso, può accadere che un segmento arrivi dopo perché nella rete ha preso una direzione diversa e arrivi con del ritardo.

Dato che in TCP non ci sono i NACK, funziona solamente il timeout: viene meno la ritrasmissione dovuta a riscontro negativo. Un problema è il valore ottimo del timeout, poiché è necessario adattarlo sulla base della stima del RTT che è variabile. TCP misura continuamente RTT per effettuare queste stime utili e trovare un valore appropriato per il timeout: se è troppo basso effettuo ritrasmissioni inutili, se è troppo alto il protocollo è inefficiente.

Viene utilizzato un filtro a media mobile: RTT viene misurato per ogni segmento non appena viene ricevuto il riscontro relativo. Non viene valutato RTT per i segmenti ritrasmessi e quello dell'ultimo segmento inviato viene detto RTT_{LAST} (ultimo RTT calcolato, non ultimo segmento di trasmissione).

- **Stima corrente (Smoothed RTT) data da**

$$RTT_{av} = (1-\alpha)RTT_{av} + \alpha \cdot RTT_{last} \qquad 0 < \alpha \leq 1$$

- **Stima della variabilità di *RTT***

$$RTT_{dev} = (1-\beta)RTT_{dev} + \beta \cdot |RTT_{last} - RTT_{av}| \qquad 0 < \beta \leq 1$$

- **Valore tipico** $\alpha = 0.125$, $\beta = 0.25$
- **Valori iniziali:** $RTT_{av} = 0$; $RTT_{dev} = 1.5s$
  - *nb: Formule al ricevimento del primo ack* $RTT_{av} = RTT_{last}$; $RTT_{dev} = RTT_{last}/2$

Se il RTT va crescendo significa che si sta creando congestione. Raccogliendo dati possiamo trovare un RTT medio (facendo una normale media): per aggiornare il RTT prendiamo questa media e usiamo la formula per lo smoothed RTT. Le formule funzionano dopo la ricezione di almeno due riscontri, altrimenti parto con RTT iniziale =0. RTT viene utilizzato per calcolare il timeout:

$$T_o = RTT_{av} + n \cdot RTT_{dev}$$

Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

Tipicamente n=2,3,4, ma può essere anche più alto. Se la rete si congestiona improvvisamente vengono ritrasmessi troppi pacchetti: il timeout viene aumentato seguendo diverse regole (fino ad una certa soglia, viene raddoppiato, ecc.)

- *Controllo di congestione*

Il controllo del flusso dati non è sufficiente a controllare la congestione nella rete: nella rete Internet attuale non ci sono meccanismi particolari atti ad evitarla, ma il compito viene delegato a TCP che essendo implementato solo negli host effettua un controllo di congestione end-to-end. Si verifica congestione quando la somma dei rate di trasmissione dei flussi che attraversano un link è maggiore della sua capacità, impedendo così la corretta trasmissione.

Il metodo più comodo per controllare il ritmo di immissione è regolare la SWND: il meccanismo si serve di una Congestion Window (CWND) che varia a seconda di alcuni eventi. In ogni istante la SWND viene dimensionata come

$$SWND = \min\{CWND, RWND\}$$

Ogni volta che viene sforato un timeout T_0 si deduce che sia avvenuto un evento di congestione: la reazione è una riduzione dell'ampiezza di CWND. Il valore di CWND viene aggiornato mediante un algoritmo, che si suddivide principalmente in due fasi, distinguibili mediante una soglia *Ssthresh*:

- Slow Start: mi trovo nel caso CWND<*Ssthresh*. CWND viene incrementata di 1 MSS (un segmento) per ogni ACK ricevuto. All'inizio della connessione Ssthresh ha un valore molto elevato e CWND=1: iniziamo la connessione sempre in Slow Start. La crescita di CWND è esponenziale, perché continua a raddoppiare le sue dimensioni. Insieme a CWND cresce anche il rate di trasmissione, stimato come

  $$R = \frac{CWND}{RTT} \quad [byte/s]$$

  L'incremento di CWND continua fino a che:
  - CWND≥Ssthresh: entro in Congestion Avoidance
  - Si verifica il primo evento di congestione, scade timeout T_0
  - CWND≥RWND: continua ad aumentare ma la finestra di trasmissione è vincolata da RWND

- Congestion Avoidance: mi trovo nel caso CWND≥Ssthresh. CWND viene incrementata di $\frac{1}{CWND}$ per ogni ACK ricevuto. Se CWND consente di trasmettere W segmenti la ricezione di tutti gli ACK relativi ai W segmenti porta CWND ad aumentare di 1 segmento: si ha un incremento lineare della finestra, che in realtà è approssimato dall'aumento unitario scritto prima.

  CWND cresce molto più lentamente rispetto alla fase di Slow Start e se smetto di ricevere ACK la finestra non cresce più.

  Quando scade un timeout di trasmissione, TCP pone Ssthresh pari alla metà della flightsize, definita come x-y+1, oppure a 2MSS, ovvero

  $$Ssthresh = \max\left\{2MSS, \frac{FlightSize}{2}\right\}$$

  Inoltre viene posto CWND=1. Solitamente $FlightSize = \min\{RWND, CWND\}$, ma resta sempre calcolata come prima.

Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)

Così facendo si trova CWND<Ssthresh e si riprende dalla fase di Slow Start, inviando per primo il segmento per il quale era scaduto il timeout (ritrasmissioni prioritarie sulle nuove trasmissioni). Il nuovo valore di Ssthresh è una stima conservativa della finestra ottimale per evitare congestione.

La versione appena vista di TCP è detta Tahoe: nella versione più recente accetta i fuori sequenza ed effettua fast retransmit (procede alla ritrasmissione dopo aver ricevuto 3 ACK uguali, senza aspettare il timeout). In un'altra versione, detta Reno, fuori sequenza e fast retransmit sono presenti, ma dopo 3 ACK uguali si pone CWND=Ssthresh+3MSS.

Document shared on www.docsity.com
Downloaded by: teocava (mattecava02@gmail.com)